Measuring what an LLM adds to malware-to-ATT&CK technique mapping

Düzgün Küçük[a,*], Fatih Ertam[a]

[a] *Department of Digital Forensics Engineering, Faculty of Technology, Fırat University, Elazığ, Türkiye*

Abstract

A large language model (LLM) pipeline for mapping malware evidence to MITRE ATT&CK (Adversarial Tactics, Techniques and Common Knowledge) techniques is mostly not the model: it wraps one in a sandbox, a disassembler, retrieval indices and rule engines, so what the model contributes is a difference against that tooling, not a score. This study measures that difference component by component, and reports the failures of the instrument that produced them. Three findings are positive. Decomposing the analysis into several model calls is worth +0.0537 F1 over a single judge on 24 real samples [+0.0362, +0.0714]; a deterministic technique assigner composing rather than choosing between its ranking and gating backends gives the cleanest identifier gate on two external corpora; and a decoding-configuration flag is worth 0.34 to 0.45 F1, more than any architecture or parameter count we measured. Four are negative or bounded. The negotiation the design was built for contributes +0.0005 at a resolution of 0.046, so the gain is the calls, not the argument; against the sandbox it is built on the full pipeline is +0.0030 F1 [−0.0353, +0.0344] at a resolution of 0.085; three retrieval components are near-inert end to end; and verbal confidence, which every deterministic gate consumes, discriminates at an area under the curve of 0.550, on an interval containing chance. Seven defects in our own instrument produced plausible results rather than errors and 2,746 passing tests caught none, the largest being that every interval in an earlier draft resampled rows from clustered observations.

Keywords: LLM agents, multi-agent systems, malware analysis, MITRE ATT&CK, evaluation methodology, silent failure, negative results

1. Introduction

Mapping malware evidence to the technique identifiers of MITRE ATT&CK (Adversarial Tactics, Techniques and Common Knowledge) is a labelling task that large language models (LLMs) are now routinely applied to, and the systems that

*Corresponding author.

Email address: `dzgnkucuk@gmail.com` (Düzgün Küçük)

perform it are rarely a model alone. A working pipeline wraps the model in deterministic tooling: a sandbox that detonates the sample, a disassembler that recovers its structure, retrieval indices that supply prior cases, and rule engines that assert techniques directly from signatures. The model's contribution is therefore a difference against that tooling rather than a score in isolation, and that difference is almost never reported. Büchel et al.'s systematisation re-evaluates more than forty technique-extraction systems in one setting and finds them "largely incomparable", because each uses its own ontology and its own inaccessible dataset [1]. An F1 value read against no baseline refers to nothing, and the baseline that matters for an LLM pipeline is the deterministic engine it was built on top of.

A second barrier is that such a pipeline is itself a distributed system, and its evaluation inherits every property of one. Each boundary between the pipeline and a server it depends on is a place where a request can succeed and still not mean what the caller assumed, and the resulting error is not visible as an error. Silent failure of this kind is a long-established class in production software [2, 3, 4] and is now being catalogued for LLM orchestration frameworks and inference engines as well [5, 6]. What changes in an evaluation setting is the artefact. The product of a silent failure is not a degraded user session that someone reports; it is a measurement that is wrong, looks right, and is written down.

This work reports the evaluation of one such pipeline, built for ATT&CK mapping over static, dynamic and network evidence, and the failures of the instrument that produced that evaluation. The system fuses six deterministic evidence layers, three domain analysts running as reasoning-and-acting (ReAct) loops over tool interfaces, an LLM judge, a weighted corroboration cascade, two retrieval tiers and deterministic confidence gating; Figure 1 shows it. It was built around four architectural claims: that multi-agent consensus over heterogeneous evidence channels beats a single reader, that a multi-layer corroboration cascade improves evidence quality, that two-tier attribution supplies useful priors, and that falsification before confidence disciplines unsupported claims. Each claim was measured at an equal total token budget against a simpler alternative, with a stochastic-noise control, with the firing rate of each mechanism measured before its effect, and with intervals resampled at the level the observations are independent at.

One of those four survives in part and three do not. Decomposing the analysis into several model calls is worth +0.0537 F1 over a single judge, but the negotiation the design was built around accounts for +0.0005 of it, so what pays is the calls rather than the argument between them. The cascade, the attribution tier and the confidence gate each return a negative or a near-null result. Alongside them the deterministic technique assigner, the one component measured against external ground truth rather than our own cohort, gives the cleanest identifier gate of its three candidate backends on both corpora it was tested on.

The study that established all of this was itself wrong several times over. Seven defects in the instrument produced plausible results rather than errors, and a suite of 2,746 passing tests caught none of them. The largest is this paper's own statistics: every interval in an earlier draft resampled rows when the observations were clustered, and correcting that widened the anchor interval 2.32-fold and turned two claims of equivalence into bounds.

1.1. Related work

Language models are already applied across the malware task surface, tracking malicious packages in a registry [7], detecting Android malware from application features [8] and from generated training samples [9], and summarising what a sample does in prose [10]. None produces ATT&CK technique identifiers from a running analysis of a Windows binary. Automated technique extraction has moved from supervised classifiers such as the Threat Report ATT&CK Mapper (TRAM) [11], rcATT [12] and TTPDrill [13], through neural extractors such as EXTRACTOR [14], AttacKG [15] and LADDER [16], to retrieval-augmented generation (RAG). TechniqueRAG [17] pairs off-the-shelf retrievers with an instruction-tuned re-ranker and fine-tunes only the generator, motivated by data scarcity: ATT&CK defines over 550 sub-techniques while roughly 10,000 annotated examples exist publicly. A hierarchical successor injects the tactic-to-technique taxonomy as an inductive bias and cuts the candidate space by 77.5% [18] [preprint], and TTPrint [19] [preprint] keeps only candidates supported by both a localised evidence span and the MITRE definition. Neither is used here as a comparator, both being unreviewed and self-reported against baselines that Büchel et al.'s systematisation calls "largely incomparable" [1]; that survey re-evaluates more than forty systems in one setting, reports a ceiling existing approaches have not crossed, and finds traditional natural language processing (NLP) approaches outperforming embedder-based and generative ones in realistic settings. Our describe-then-map design removes technique-identifier recall from the model entirely, which is a domain instantiation of Infer-Retrieve-Rank [20] [preprint] and is stricter than [17], where the model still ranks among retrieved candidates. That the retrieval backends are separable on ranking and on gating is likewise not ours, Abdallah et al. [21] treating retrieval confidence as a dimension distinct from effectiveness and finding calibration "consistently weak across model families"; Section 3.2 measures the separation on TRAM2, the corpus that tool defines [11], and on the independently annotated AnnoCTR corpus [22]. Two adjacent results have prior art we cite rather than compete with: that a false correction costs more than a missed one is why grammatical error correction evaluates with F0.5 [23, 24], and neural text degeneration has competing established accounts locating the cause in the likelihood objective and in training-data repetition [25, 26].

Agentic systems now drive reverse-engineering backends through tool interfaces, and the model tier splits by role: fine-tuned binary specialists are small and open-weight, among them ReCopilot [27] [preprint], LLM4Decompile [28] and Nova [29], while tool-driving agents use frontier models. TTPDetect [30] [preprint] is the closest system to ours, identifying tactics, techniques and procedures in stripped malware binaries through retrieval-narrowed entry points and a function-level agent, reporting 93.25% precision and 93.81% recall at function level and 87.37% precision on real-world malware; we do not claim binary evidence as an input modality, and that ground is theirs. Degradation of tool selection as a catalogue grows is documented generally, with LongFuncEval reporting a 7 to 85% performance drop as tool count increases [31] [preprint] and a cut from 46 to 19 tools saving up to 70% of execution time [32], but not for a reverse-engineering catalogue whose tools semantically overlap. Confidentiality-driven local deployment is likewise established: REx86 [33] fine-tunes eight open-weight models for x86 reverse engineering because "cloud-hosted, closed-weight models pose privacy and security risks and cannot be used in closed-network facilities", validating with a 43-participant user study.

Multi-agent debate is a standard pattern, including in security, where PhishDebate [34] assigns four specialised agents to distinct views of a webpage and reports 98.2% recall. Two recent results change how such claims must be read. Holding the reasoning token budget constant, Tran and Kiela find single agents consistently match or beat multi-agent systems on multi-hop reasoning, with an argument from the Data Processing Inequality and a crossover appearing only at degradation 0.7 [35] [preprint]. Bertalanič and Fortuna [36] reach the same conclusion on 7B and 8B open-weight models, where debate consumed 2.1 to 3.4× more tokens for equal or lower accuracy, and name sycophantic conformity, contextual fragility and consensus collapse as its failure modes. Both scope their negatives to homogeneous agents decomposing a single context and both name degraded or noisy context as the exception, which is the regime a 600-import binary plus a sandbox report plus a network capture occupies; whether heterogeneous evidence-channel decomposition survives a control that homogeneous debate fails is the question our equal-budget ablation asks. Weighting evidence by source reliability and discounting non-independent sources is Dempster-Shafer theory, decades old, and our cascade is an instance of it with hand-chosen constants; measuring five perturbations of those constants, including inverting the most- and least-trusted layers, changed the top-10 ranking on 10.6 to 27.5% of samples and the corroborated set on none, because the corroboration decision is a function of layer count alone.

This work's methodological results belong to an established critique tradition. Arp et al.'s *Dos and Don'ts* identified ten pitfalls in machine learning for security, and *Chasing Shadows* [37] is its direct successor, surveying all 72 peer-reviewed LLM-security papers of 2023 and 2024 and finding every one exposed to at least one of nine pitfalls with only 15.7% explicitly discussed. A published negative RAG result in malware analysis exists by a mechanism different from ours, retrieved context diluting a signal-extraction task [38]; comparison against a trivial label-only baseline is long established [39]; and a systematic review by the same group found a considerable share of published multi-label results no better than that baseline [40] [preprint]. On temporal stability the axis matters more than the result: performance falls as evaluation moves away from the training period and scale does not close the gap [41], but that work varies the distance between training and test time while ours fixes the model and varies the era of the input binary. We no longer have a result on that axis, for the reason given in Section 3.10.

The paper's methodological spine belongs to an area that already exists. Gunawi et al. [2] measured where an error signal is lost, finding that 13% of propagation channels drop an error code without handling it. Yuan et al. [3] ruled that a handler which only logs is a handler that ignores, and attributed 25% of catastrophic failures to errors explicitly signalled and then ignored. Alquraan et al. [4] found the majority of network-partition failures in cloud systems to be silent. The LLM-specific instance is now being catalogued, in bug taxonomies for agent orchestration frameworks [5] and for inference engines [6], the latter carrying both incorrect request parsing as a root cause and silent error as a symptom; silently overruling a requested parameter back to a default is itself a named misconfiguration failure mode with a measured prevalence [42]. Adjacent work reaches the same place from the interface side: mining defects in Model Context Protocol (MCP) reference servers, Oyelayo et al. [43] find data validation and type issues to be the second most prevalent class at 13.97%, the bucket our null-for-unset-optional defect falls into, and input dependencies that a machine-readable application programming interface (API) specification cannot express are "the norm, rather than the exception, with 85% of the reviewed APIs" [44]. The detector is not new either: "distinct inputs should produce distinct outputs" is an ordinary metamorphic relation, and metamorphic testing exists precisely for programs whose correct output is unknown [45]; stress-testing an LLM judge's consistency under input variation is likewise established [46]. Nor is the newest of our mechanisms: our own parameter-size series reproduced the ρ=0.85 at p=0.014 the nearest prior work reports [47] [preprint], from an axis on which the low-scoring row was a model disabled by a configuration flag rather than limited by its size. That phenomenon is established: under strict output-length constraints performance "might not scale monotonically with the model size" [48], across 6.5M instances inference-time configuration moves performance "both absolute and relative" [49], extra test-time compute lets a smaller model overtake a much larger one [50, 51], and lengthening reasoning steps improves accuracy [52]. The same disagreement exists one layer down, where OpenAI-compatible servers differ over which token-limit parameter they honour, documented as `llama.cpp` #8634 [53] [preprint] and `vLLM` #11976 [54] [preprint].

Every result of ours cited above was produced on one model on one machine, except where a statement concerns retrieval or the deterministic cascade and no model was involved at all. That matters here because parameter size was the only statistically significant predictor of F1 in the nearest external study [47] [preprint].

1.2. Motivation and contributions

Two observations motivated this work. First, a component of an LLM analysis pipeline is ordinarily justified by the mechanism it implements rather than by a measurement, and a mechanism sound in isolation can contribute nothing once wired to real inputs; establishing which holds needs an equal-budget comparison against a simpler alternative and a baseline containing no language model at all. Second, such a comparison is only as trustworthy as the instrument producing it, and in a pipeline assembled from several servers that instrument fails without announcing itself. The contributions are therefore organised around the evaluation rather than the architecture, and each is a measurement or a mechanism rather than a proposal.

(i) A deterministic technique assigner, measured on two external corpora. The model never emits a technique identifier; a retrieval index assigns it. Ranking quality and gate quality turn out to be separable axes on which no single backend wins, and composing the per-axis winners gives the cleanest identifier gate of the three on both TRAM2 and AnnoCTR, with all three orderings replicating across corpora annotated by different people for different tasks (Section 3.2).

(ii) What decomposition is worth, and what it is not worth. On 24 real samples, decomposing the analysis into several model calls beats a single judge by +0.0537 F1 [+0.0362, +0.0714]. A stochastic-noise control gains the same +0.0532, and the contrast that isolates the negotiation returns +0.0005 at a resolution of 0.046, so what pays is the calls the arms share rather than the argument between them. This is the difference between knowing that a design works and knowing why.

(iii) A no-LLM baseline for ATT&CK mapping, and the pipeline scored against it. The sandbox the pipeline is built on maps its own signature hits to technique identifiers deterministically, and scored per sample against family-level MITRE ground truth it reaches F1 0.1526 [+0.1114, +0.1998] on 97 samples. On the samples where both arms ran, the full pipeline is +0.0030 F1 above it [−0.0353, +0.0344], at a resolution of 0.085. This is the comparison without which every F1 in this literature, including our own earlier ones, refers to nothing.

(iv) Three architectural claims that did not survive, each with the resolution of the design that tested it. A weighted corroboration cascade, two-tier attribution and a deterministic confidence gate were each measured and each returned a negative or a near-null result. A null without its minimum detectable effect is unreadable, so every one is stated with the effect its design could have detected at 80% power. The configuration axis is where the effects are: a decoding flag is worth 0.34 to 0.45 F1, more than any architecture or parameter count we measured, and an arm can be labelled matched while running the opposite configuration because one provider accepts the parameter without acting on it.

(v) Seven instrument failures, with the layer each occurred at and what actually found it. Each produced a plausible result rather than an error, and 2,746 passing tests caught none. Four crossed a boundary with another server and fall inside a described class of silent failure; three crossed no boundary at all and share a different shape, in which a measurement's cause was assigned rather than measured. What found four of them was one line of arithmetic: how many distinct outputs N inputs produced.

2. Materials and methods

2.1. Scope and assumptions

This study evaluates one pipeline that maps evidence about a Windows Portable Executable (PE) malware sample to MITRE ATT&CK technique identifiers and emits a Structured Threat Information Expression (STIX) bundle. The scope is narrow in four respects and every result should be read inside it. The model is a single local mixture-of-experts deployment, quantised, on one host, with three commercially hosted services used only as comparison endpoints. Ground truth is family-level MITRE `uses`, resolved to a sample through its family signature, which is coarse and carries a structural recall ceiling. Only Windows PE is analysed dynamically, because the sandbox instance has one analysis machine and no Linux guest.

2.2. *The pipeline under evaluation*

Figure 1 shows the system. Its organising rule is that the model proposes and a deterministic layer disposes: the model describes behaviour and never emits a technique identifier or a final set, and every identifier that reaches the analyst has been assigned, gated and reconciled by code. That is why the deterministic layer dominating the output is the design working rather than a component failing, and why what has to be measured is what the model adds on top of it. Six deterministic evidence layers assert techniques directly from signatures, rules and byte markers, with no model involved. Three analysts then run as ReAct loops over heterogeneous evidence channels, each bound to one tool server; a negotiation node tests for consensus and routes disputes to a revision pass; a judge synthesises a verdict and emits STIX 2.1; and a deterministic reconciliation and gating stage adjudicates between the judge's bundle and the corroboration cascade before the analyst sees anything. The seven instrument failures of Section 3.7 are marked on the figure at the boundary each occurred at, because the claim that four crossed a boundary with another server and three did not is a claim about this topology.

The cascade scores a technique t by weighting each contributing layer's confidence by how far that layer is trusted, then lifting the result by how many layers agree:

$$s(t) = \min\left(\frac{\sum_{d \in D(t)} w_d \, c_d(t)}{\sum_{d \in D(t)} w_d} \; m(|D(t)|), \; 1 \right) \tag{1}$$

$D(t)$ are the domains claiming t and $c_d(t)$ is domain d's confidence for it, a mean over a model's claims and a maximum over a rule's. The trust weights w run from 0.90 for YARA to 0.20 for network evidence, and the corroboration multiplier m rises from 1.00 at one contributing layer to 1.90 at 5.

Two design choices are not what failed. Tool exposure is not tool availability: the reverse-engineering server advertises ~165 tools, exposing the full catalogue to a model with roughly 3B active parameters is infeasible because tool selection degrades before the context does, and the static analyst is therefore given a curated allowlist of 20 tools, with high-value tools invoked deterministically from code. And the pipeline degrades rather than fails when a service is absent, so with the sandbox unreachable the dynamic path is skipped and the run completes on static evidence, a behaviour pinned by a test. That mattered during the study, because the sandbox was reachable from one network only and every static-only result reported here was produced by that degradation working as designed.

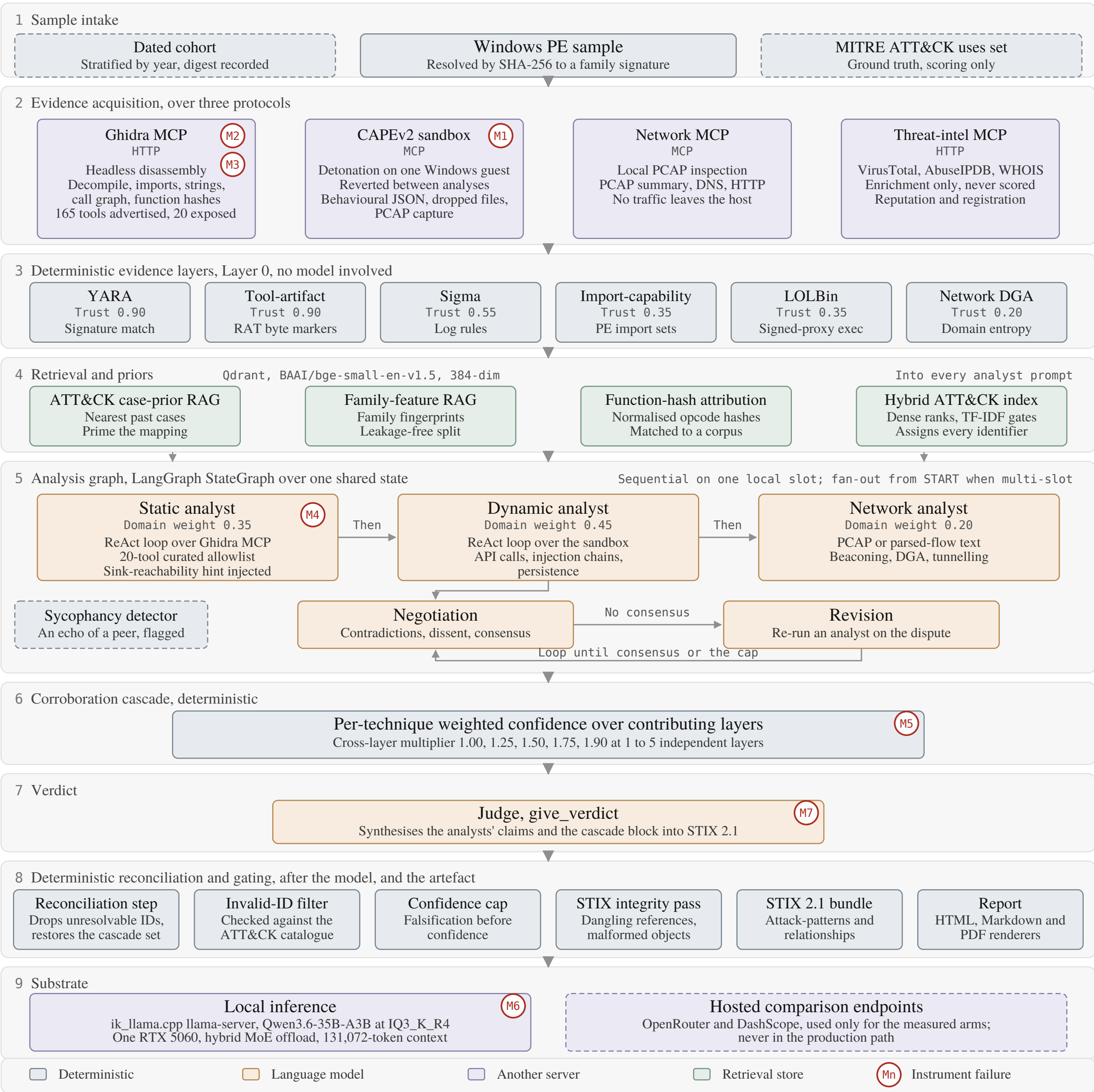


Figure 1: The pipeline under evaluation, with each of the seven instrument failures of Section 3.7 marked at the boundary it occurred at. Trust weights and cross-layer multipliers are the shipped constants.

2.3. *Samples, ground truth and the sandbox*

The dynamic cohort is n=100 Windows PE samples from MalwareBazaar, stratified by year at seed 20260810 with its digest recorded, of which 97 produced a usable sandbox report. Ground truth resolves a family signature to an in-repo MITRE `uses` fixture through an alias map with a CamelCase-split fallback. It is family-level and uncurated at technique level, so a single binary of a family need not exhibit all the techniques catalogued for that family and absolute recall carries a structural ceiling. Absolute F1 values here are therefore not comparable to work using per-sample expert labels; the bias is approximately constant across arms, which makes within-study contrasts the defensible reading. The withdrawn temporal-drift study's manifest survives as 210 dated samples over 7 cohorts and 27 families, metadata only.

Three properties of the sandbox bound what can be reproduced. It is CAPE 2.5, the Config and Payload Extraction sandbox, with one analysis machine and no Linux guest, so an Executable and Linkable Format binary submitted there joins the 10,348 tasks never scheduled. Reports are retained for days rather than indefinitely, with 18 of 18 sampled older tasks reporting no report directory, so the local archive is the only copy and re-submitting the same samples is not a reproduction recipe on its own. And the instance rate-limits with a throttle response byte-identical to its auth refusal, `{"error": true, "message": ""}` in both cases, so a harness reading that string as a capability verdict will mis-classify a throttle as a missing permission, as ours did briefly. A new submission is scheduled in ~1 second and completes in ~5.5 minutes, putting an n=100 cohort at roughly 9 hours of sandbox time.

2.4. *Models, endpoints and serving configuration*

Table 1 identifies the local model and inference engine by Secure Hash Algorithm (SHA-256) digest and by source commit. Naming the imatrix calibration dataset matters, because two IQ3 quantisations of the same model calibrated differently are not the same artefact. The engine commit was reconstructed rather than recorded: `llama-server --version` prints `version: 0 (unknown)` on a build tree that was never a git checkout, so the commit was recovered from a vendored depth-1 clone and proved against an identical 837-file source list, one file differing only in line endings.

```
llama-server -m Qwen3.6-35B-A3B-IQ3_K_R4.gguf \
  -c 131072 -t 16 -fa on -ctk q8_0 -ctv q8_0 -ngl 999 \
  -ot "blk\.([1-3][0-9])\.ffn_(up|gate|down)_exps=CPU" \
  --context-shift on --jinja --host 0.0.0.0 --port 8080
```

Restarting the server between arms changes no decoding parameter and is therefore measurement-neutral.

Table 2 lists the endpoints each arm ran against, and one parameter on it decides more than the model names do. `enable_thinking` is honoured on DashScope and ignored on OpenRouter, not rejected but accepted, recorded in the result file, and not acted on: the re-run requesting it returned a 56.2% reasoning share against

Table 1: The local model and inference engine, pinned by digest and commit, and the four measured host properties that bound what can be reproduced on it.

model	`Qwen/Qwen3.6-35B-A3B`, quantised IQ3_K_R4 by Unsloth
model file SHA-256	`d0de70ef…c4ea`, computed locally and matching the HuggingFace etag
HuggingFace revision	`cfd350fd…1f0d`, retrieved 2026-05-11
imatrix calibration	`unsloth_calibration_Qwen3.6-35B-A3B.txt`, 510 entries over 76 chunks
architecture	hybrid recurrent (state-space-model keys + `full_attention_interval=4`), from the model file metadata
engine	`ik_llama.cpp` at `eb570eb96689c235933b813693ca28ab9d3d26de` (#1778)
host	one RTX 5060, 31 GiB random-access memory, 16 threads
server growth	10.4 GB fresh, 14.8 GB after one analyst pass, 18.1 GB after 60 minutes; anonymous and unreclaimable, so long paired runs restart the server between arms
context	131,072 of the native 262,144, 30 blocks on the central processing unit; serving 64k instead does not lower the peak (14.6 against 14.8 GB)
generation rate	162 to 20 tokens/s, because the engine writes and erases a 63 MiB recurrent-state checkpoint every few hundred tokens at ~39k context
disassembly container	capped at 6 GB, its Java virtual machine flag `-Xmx4g` bounding the heap only, measured resident set size 5.15 GB against that nominal 4 GB cap

56.5% without it, while on DashScope the same request produces 0.0%. The flag is worth 0.34 to 0.45 F1 on the two models where it can be set, so an arm can be labelled matched while running the opposite configuration, and arm selection therefore keys on the measured reasoning share of a completed arm rather than on the flag requested.

Two serving details matter to anyone reproducing an arm. The local server's output cap must be sent twice, because `ChatOpenAI(max_tokens=N)` reaches the wire as `max_completion_tokens`, which `ik_llama.cpp` does not read, so the cap must also travel in `extra_body`; measured on this host, 48 requested yields 2805 generated with the renamed field alone and 48 with both. And that server truncates silently, returning `finish_reason: "stop"` at the cap and never `"length"`, so telemetry detecting truncation by finish reason reports zero however often the cap binds. The frontier arm ran on a free tier allowing 50 requests per day at 20 per minute, and the first attempt exhausted it mid-flight with 9 calls landing and 16 returning HTTP 429, the Hypertext Transfer Protocol (HTTP) status for a rate limit. One call per sample over a 97-sample cohort is two days at that rate, and a full-pipeline arm at roughly 20 model requests per analysis is not reachable on that tier

Table 2: The inference endpoints each arm ran against, with the configuration measured rather than the configuration requested.

arm	endpoint	model	reasoning	n
local baseline	`ik_llama.cpp`, this host	Qwen3.6-35B-A3B (IQ3_K_R4)	off	25
frontier	OpenRouter	`nvidia/nemotron-3-super-120b-a12b:free`	on, not controllable	25 ×2
same weights, hosted	DashScope	`qwen3.6-35b-a3b`	off	25
same weights, hosted	DashScope	`qwen3.6-35b-a3b`	on	25
larger, hosted	DashScope	`qwen3.6-plus`	off / on	25 each

at all.

2.5. *Experimental design*

Four terms are used in one sense throughout. An *arm* is one condition of one experiment, a configuration under which every input is scored, so that two arms differ in the one thing being tested. A *sample* is a real malware binary from the dated cohort, scored against its family's MITRE `uses` set. A *fixture* is one of the five synthesised inputs used by the equal-budget arms, whose evidence is generated from its own technique list and which therefore has a ceiling a real sample does not; samples and fixtures are different populations with different ground truth, they are never pooled, and where both appear in one figure they appear on separate axes. The *reconciliation step* is the deterministic pass that compares the judge's bundle against the corroboration cascade's set and restores what the judge omitted.

Arms are compared at an equal total token budget: a multi-agent arm making N calls receives B/N per call against a single arm's one call of B, so a win cannot come from spending more. The design follows Bertalanič and Fortuna [36], including a stochastic-noise control in which one analyst receives evidence irrelevant to the sample. That control establishes that the harness can detect a difference at all: in the fixture consensus study the noise arm separated (0.3366 against 0.3975 and 0.4136) while the treatment did not, which distinguishes no effect from no sensitivity. For reasoning models the budget must cover reasoning, because our frontier endpoint spent 56.5% of its output on reasoning across 25 real prompts and 84% on a one-token answer, so capping content alone would have granted it roughly twice the generation for the same nominal budget. Arms must also sit on the same side of the reasoning switch: measured on two models at 25 calls each, enabling the flag moved F1 by 0.45 on one and 0.34 on the other, in both cases because the model spent its entire output budget thinking and returned no answer, with 24 of 25 calls hitting the cap on the second. An unmatched pair therefore does not produce a noisy comparison; it produces a comparison of the flag.

Three further rules were adopted after each one caught something, and all three are cheap. The first is that a mechanism's firing rate is measured before its effect: the confidence cap fires on 0.82% of techniques, so an ablation would describe the 99.2% where the cap never ran, while the sink-reachability hint fires on 56.7% of samples and an ablation of it is a real statement about the hint. The cap's preconditions explain its rate, since it applies to three technique families only when the sole contributing layer is static and 84% of those claims are YARA-only. An ablation must therefore be restricted to the samples where the mechanism fires, because averaging a feature over inputs it never touched dilutes a real effect toward zero. The second is that the number of distinct outputs the N inputs produced is counted and reported as a result column rather than as a test, because the signal exists only in the batch while each individual response is well-formed; we report 63 distinct call-graph sizes across 97 samples. The third is that per-sample outputs are retained for every study, since aggregates cannot be re-interrogated: when a defect was found that might have affected a completed 210-sample study, the question could not be asked because only the summary survived. Long-lived servers accumulate state that crosses sample boundaries, so the reverse-engineering server is restarted between samples, which recovered 12 of 14 samples previously recorded as unmeasurable. Two audits were added for the same reason: diffing the claim register against the evidence log, and counter-searching each novelty claim in an adjacent field's vocabulary, which demoted five including the one central to this paper. A counter-search is recorded whether or not it demotes anything, because a searched absence is only credible if the looking is documented.

2.6. Statistical analysis

Every arm is scored on the same samples in the same order, differences are computed per sample and then aggregated, and 95% bootstrap confidence intervals (CIs) are taken over the paired deltas. The observations are nested, with many claims from one sample and many samples from one family, so the bootstrap resamples whole clusters, the p-value comes from flipping the sign of whole cluster means rather than of rows, and each interval is reported with the intraclass correlation coefficient (ICC) of its measure and the effective sample size that implies. Two properties of that test are reported with it. With k clusters the two-sided version cannot return a p below

$$p_{\min} = 2/2^k \tag{2}$$

which at k=5 is 0.0625 and puts α=0.05 out of reach whatever the effect size; and above 20 clusters the 2^k assignments are sampled rather than enumerated, at 20,000 draws with the observed assignment counted in its own reference set, so the smallest p that route can return is one over the draws rather than zero. Each reported p names which route produced it. The two properties pull in opposite directions: raising the cluster count lifts the floor and is what makes enumeration stop being possible, which is why the consensus arms were re-run one sample per family. Multiplicity is controlled with Benjamini-Hochberg over two declared families, and every null is reported with the minimum detectable effect of the design that produced it at 80% power, because a null without its resolution reads either as equivalence or as a failed experiment depending on the reader.

Both choices were corrections rather than plans: every interval in an earlier draft resampled rows rather than clusters, and an early study ranked prompt structures on a single-run parsed-claim count whose ranking inverted when the decoding budget changed. No result in this paper rests on a single run.

3. Results and discussion

3.1. The no-LLM baseline and the pipeline measured against it

Every accuracy number here is read against one reference: the sandbox the pipeline is built on, mapping its own signature hits to ATT&CK technique identifiers deterministically, with no language model anywhere. Table 3 reports it with the like-for-like comparison against the pipeline.

Table 3: The no-LLM baseline and the pipeline against the engine it is built on. Both blocks are scored per sample against family-level MITRE ground truth by the same code; intervals resample families, not samples.

	mean	95% cluster CI	ICC	effective n
baseline alone, n=97 samples over 24 families				
precision	0.2413	[+0.1850, +0.3010]	0.584	35.0
recall	0.1328	[+0.0896, +0.1810]	0.638	33.0
F1	0.1526	[+0.1114, +0.1998]	0.692	31.2
paired, n=13 samples over 8 families, F1				
CAPE alone, no language model	0.1130	[+0.0654, +0.1624]		
pipeline, static evidence only	0.1130	[+0.0770, +0.1543]		
pipeline, with the sandbox report	0.1160	[+0.0823, +0.1485]		

Ground truth is resolved per family, so two binaries of one family are scored against a byte-identical label vector and are not two observations. Resampling samples rather than families gave an F1 interval 2.32 times narrower on data whose intraclass correlation is 0.692: the row count is 97 over 24 families at mean 4.04 samples per family, and the count supporting an inference is 31.2. The sandbox asserted at least one technique on every sample, so this is a predictor rather than an artefact of an empty one.

Three analyst agents, a negotiation loop, a revision pass, an LLM judge and a weighted corroboration cascade land +0.0030 F1 from the signature engine they are built on top of, 95% cluster CI [−0.0353, +0.0344], exact cluster sign-flip $p = 0.9609$, better on 7 of 13. The samples differ individually in both directions, so the system is not reproducing the sandbox's answers but reaching different answers of the same quality. At 8 families the design could detect 0.085 F1 and did not,

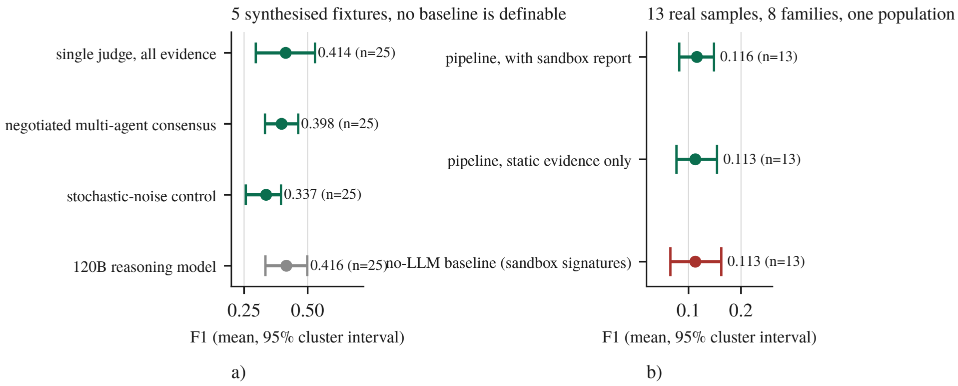


Figure 2: Every measured arm on one F1 axis. a) the synthesised fixtures, b) the real-sample population. Points are means over per-sample F1, bars are 95% intervals resampling the cluster the observations are independent at.

which bounds the pipeline's contribution rather than showing it is zero. The cohort is 97 rather than 100 because 56 of the submitted batch never ran while the sandbox reported them complete, the fifth boundary case of Section 3.7.

The two panels of Figure 2 do not share an axis. Panel a)'s arms run on 5 synthesised fixtures scored against per-sample truth while the baseline runs on real binaries scored against family-level truth, and no baseline is definable there at all since a regular expression over the dictionary that generated the evidence scores 1.000 on all 5 of them by construction. On one scale a no-LLM baseline near 0.1526 beneath treatment arms near 0.4136 reads as a large gain for the pipeline, and it is not one. Panel b) is the comparison that can be made, and its three intervals overlap.

3.2. The deterministic technique assigner

The pipeline never asks the model for a technique identifier. The analyst describes behaviour and a retrieval index over the official ATT&CK corpus assigns the identifier, which removes identifier recall from a model that does not have the taxonomy memorised and cannot be made to by prompting. That assignment is the one component measured against external ground truth rather than against our own cohort, on two public corpora neither of which is the ATT&CK text the index is built from (Table 4).

Gate quality is the margin the absolute threshold has to work with, the mean retrieval score of the top-ranked technique when it is the right one less that score when it is not:

$$g = \mathrm{mean}(s_1 \mid \mathrm{correct}) - \mathrm{mean}(s_1 \mid \mathrm{wrong}) \tag{3}$$

Ranking quality and gate quality are separable axes and no backend wins both. The semantic index ranks better on both corpora, 0.392 against 0.329 top-3 on TRAM2 and 0.214 against 0.183 on AnnoCTR, while its scores barely separate a correct pick from a wrong one, at gate +0.019 and +0.062, because everything it retrieves sits near the same similarity. The lexical index ranks worse and gates cleanly, scoring

Table 4: The technique assigner on two external corpora: TRAM2 [11], sentence classification, and AnnoCTR [22], expert entity linking in running report prose. MRR is the mean reciprocal rank of the correct technique; gate separation is Equation (3).

	TRAM2, n=4,913			AnnoCTR, n=3,289		
backend	top-3	MRR	gate	top-3	MRR	gate
lexical	0.329	0.274	+0.085	0.183	0.147	+0.135
semantic	0.392	0.319	+0.019	0.214	0.176	+0.062
hybrid	0.392	0.319	+0.115	0.214	0.176	+0.168

near zero for unrelated evidence. Composing the per-axis winners matches the semantic index's ranking and produces the cleanest gate of the three on both corpora, +0.115 and +0.168. That composition is the shipped default, and the gate is what makes the assigner safe to leave authoritative: an identifier it cannot align is refused, not guessed.

All three orderings replicate across the two corpora, which is the claim. Absolute accuracy does not and is not compared: entity linking in unconstrained prose over a 691-technique space is harder than sentence classification, and 48 AnnoCTR labels fall outside our ATT&CK bundle. The direction is not ours either: Büchel et al. [1] report traditional NLP approaches outperforming embedder-based ones in realistic settings, a stronger and earlier form of the gating half. What this adds is that the two axes come apart and that composing them beats choosing between them.

That layer also emits the artefact, and its conformance is measured with a standard external validator rather than with the integrity pass this project wrote itself. The OASIS cti-stix-validator 3.3.1 finds 0 errors and 0 referential warnings in a bundle built from the production models. Two defects the integrity pass had no opinion about were found that way: every object lacked the `spec_version` property STIX 2.1 requires on each STIX Domain Object rather than on the bundle, so no bundle the project had emitted was identifiable as STIX 2.1, and `null` and empty-array properties were serialised as present, which the specification forbids. Both are fixed. The repair pass is not what makes a bundle conform: across 4 bundles with injected defects it recovers 0 to validator-clean, and what it does do is preserve 53 objects that rejection would discard.

3.3. Multi-agent consensus at equal budget

Three arms at an equal total token budget, including a stochastic-noise control in which one analyst receives irrelevant evidence, run first on the fixtures and then again on real binaries one per family so that every cluster holds a single observation (Table 5).

On the fixtures neither contrast is readable. The negotiated arm returns −0.0161, [−0.1247, +0.0819], at 3.2× the tokens, two orders of magnitude below the design's 0.223; the noise control moves −0.0770, [−0.1337, −0.0233], with all 5 samples in

Table 5: The equal-budget consensus arms, on synthesised fixtures and on real binaries. The real-sample block is scored against the same family-level ground truth as Table 3; the fixture block is not comparable to it.

arm	mean F1	rows	output tokens	× single
5 synthesised fixtures				
single judge, all evidence at once	0.4136	25		
negotiated multi-agent consensus	0.3975	25		3.2
noise control	0.3366	25		
24 real samples, one per family				
single judge	0.0967	1	39,986	1.00
negotiated consensus	0.1503	4	110,395	2.76
noise control	0.1498	4	94,388	2.36

the same direction, also below its own 0.120, and the exact permutation test returns p = 0.0625, the smallest available, because at 5 clusters the two-sided sign-flip test floors at 0.0625. An earlier draft read that separation as proof of sensitivity; what it shows is that all samples moved the same way.

The real-sample replication is where the design can see, the sign-flip floor at 24 clusters being 0.00005. Both multi-agent arms beat the single judge and neither beats the other: negotiated minus single is +0.0537, [+0.0362, +0.0714], q = 0.0001, better on 19 of 24 families, while noise minus single is +0.0532, [+0.0235, +0.0833], q = 0.0040. The contrast that isolates the mechanism returns +0.0005, [−0.0294, +0.0312], q = 0.9751, on a design resolving 0.046 F1, and this time that means something: the effect the negotiation would have to carry, the +0.0537 its neighbours show, is above that resolution, so if the negotiation were doing the work this design would have seen it. What separates the multi-agent arms from the single judge is therefore what they share, 4 model calls instead of 1: spending more calls on a sample helps, making them argue does not, at this budget on this corpus with this model.

The two corpora gave opposite signs, which is the sharpest evidence we have about the fixture corpus. On fixtures the noise control ran −0.0770 against the single judge and was worse on every sample; on real binaries it runs +0.0532 and is better, at the same q as the negotiation. Nothing about the arms changed and the evidence did. A corpus whose inputs are generated from its own answer key does not merely have less power than one of real binaries; it can rank the arms the other way round.

3.4. Model scale, quantisation and the reasoning flag

Table 6 reports the local model against a 3.4× larger frontier model on the same fixtures at the same 2,400-token output budget, and against the same weights hosted at full precision with and without the reasoning flag.

Two of these three comparisons are bounds the design cannot read, and are reported as such. The frontier model lands within three thousandths of F1 of a 35B model quantised to run on one desktop graphics processing unit, better on

Table 6: Model scale, quantisation and the reasoning flag, all paired against the local arm on the same 5 fixtures and repeats. The local arm is Qwen3.6-35B-A3B at IQ3_K_R4, the frontier arm Nemotron-3-Super-120B-A12B, and the vendor-hosted arms the same Qwen weights at full precision.

arm	think	F1	vs local	95% CI
local, 3-bit	off	0.4136		
frontier, 3.4× larger	on	0.4162	+0.0026	[−0.1322, +0.1460]
frontier, flag requested	on	0.4149	−0.0014	[−0.0695, +0.0799]
vendor-hosted, full	off	0.3507	−0.0629	[−0.2133, +0.0963]
vendor-hosted, full	on	0.0080	−0.4056	[−0.5232, −0.2880]

12 and worse on 13 of 25 rows, against a minimum detectable effect of 0.300 F1 that is coarser than every architectural effect reported here. It is confounded as well: the local arm runs with its reasoning stream disabled, necessarily, because the local server otherwise returns an empty answer, while the frontier arm spent 56.5% of its budget reasoning, and re-running it with the parameter set returned 56.2%. The arm was not a collapsed one, having parsed 25 of 25 calls with no refusals and 1 hitting the output cap; an earlier version of it reported 0.5025 at n=9 and suggested a lead that did not survive completing the sample. We report the null because the literature's prior predicts otherwise, parameter size being the only statistically significant predictor of ATT&CK-classification F1 on the nearest task [47] [preprint], and we cannot test that as a series either, because a rank correlation over model size needs the arms matched on the flag worth more than the size effect and the only endpoint above 35B does not permit that match. Quantisation is the second bound: the paired difference is −0.0629 on [−0.2133, +0.0963] at n=25, admitting a penalty of up to 0.2133 F1 against a minimum detectable effect of 0.331, so 3-bit quantisation is not demonstrably this pipeline's handicap rather than not one.

The third comparison returns the largest effect measured anywhere in this paper. The reasoning flag replicates at +0.3427 paired ([+0.2008, +0.4936]) on the model we host, with 24 of 25 calls exhausting the output budget on reasoning, and independently on a third endpoint, which with reasoning enabled spent 99.9% of every call's budget thinking and scored 0.0000 across 25 calls against 0.4501 with it disabled, a difference of +0.4501 F1, [+0.3604, +0.5437]. The flag is the only effect in this paper that exceeds its own design's minimum detectable effect, at 0.310 here and 0.200 on the third endpoint: the largest effect measured on any arm is a configuration flag, not an architecture and not a parameter count. Two qualifications belong with it. The comparisons are post-hoc, run after the confound was discovered, and corrected as a family of 5 their q-values are 0.1042 and 0.1042; and like every comparison on this corpus they sit at the exact test's floor of 0.0625. On its shortest answer the frontier arm spent 77 of 92 output tokens thinking.

3.5. Retrieval, confidence and firing rate before effect

Three retrieval components were built independently, each works where it was built, and each moves nothing where it is wired in (Table 7).

Table 7: Three retrieval components, each measured in isolation and again end to end.

component	measured in isolation	contribution end to end
ATT&CK case-prior RAG	self-consistency 0.620 against a 0.424 frequency prior, scored on the corpus's own prior attributions rather than on MITRE ground truth	loses to the frequency prior in production (0.111 against 0.123, MITRE ground truth)
family-feature RAG	yes, recall@5 0.199, 6.3× chance, leakage-free split	+0.0029 F1, precision −0.0088, n=19
opcode-hash attribution	n/a, never fires	0 of 18 samples; 7,716 functions hashed, 0 matches

The first row's isolation figure is not accuracy, because the leave-one-out corpus is labelled with the pipeline's own prior attributions; it is retained because a frequency prior scored the same way reaches 0.424, and because the production column beside it is scored against MITRE ground truth on held-out samples. The three fail for three different reasons. The first fails on vocabulary mismatch between query and corpus, visible only in the score distribution while top-k accuracy stayed healthy. The second is simply small. The third fails twice over: its corpus holds 3 samples under 2 generic labels, and the 8-instruction floor that correctly excludes thunks leaves 9 of 18 real samples with one hashable function or none.

Verbal confidence, which the cascade and every deterministic gate consume, discriminates correct from incorrect claims at an area under the curve of 0.550, near chance, concentrated on 4 round values. Resampling samples rather than claims puts the interval at [0.475, 0.671], which contains chance, and 3 of the 5 samples sit at or below chance individually, with the pooled figure carried by the one reaching 0.778. Figure 4 shows why: 186 of the 210 claims carry confidence exactly 1.0, so the score has almost no ordering to contribute. The estimate is rank-based with ties averaged, which here is not a detail: sorting the tied claims arbitrarily returns 0.458, a difference entirely an artefact of how 89% of the data is handled. This matches Kumaran's finding [55] [preprint] that verbal confidence measures willingness to commit rather than correctness, so every deterministic gate keyed to that number is keyed to something we cannot distinguish from noise.

Whether any of these mechanisms can be ablated at all depends on how often it engages, which Table 8 and Figure 3 report. Three of the four mechanisms the architecture was built around engage on almost nothing, so an ablation of any would return a null describing the cases where the mechanism never ran. Only the sink-reachability hint clears a rate at which an ablation carries information, which is why it is the one we ablated.

The hint does nothing measurable. On n=6 pairs it is better on 2, worse on 2 and tied on 2, with per-pair deltas of +9, +4, 0, 0, −4, −6, large in both directions

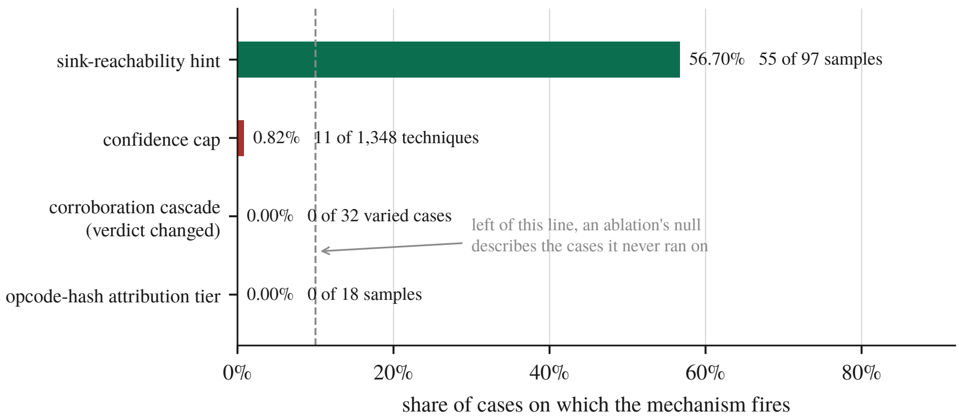
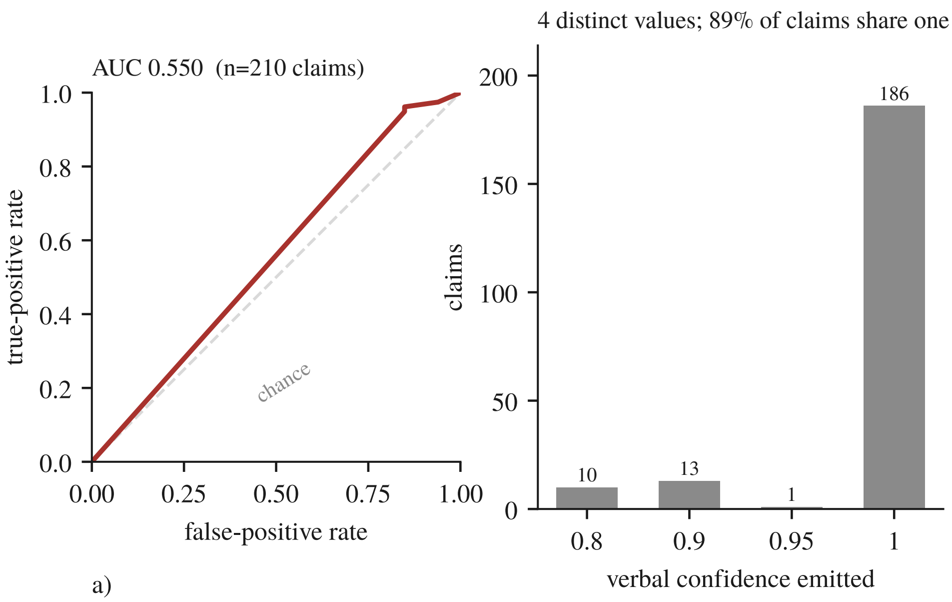


Figure 3: Firing rate decides whether an ablation can be read at all.



a)

b)

Figure 4: The number the gates consume barely orders anything. a) the receiver operating characteristic over 210 claims against resolved ground truth, b) the confidence actually emitted.

Table 8: Firing rate before effect, and the one ablation it justified. Units differ per row of the lower block and are named in the row.

mechanism	fires on	consequence
confidence cap	0.82% of techniques	an ablation would measure nothing
sink-reachability priority hint	56.7% of samples (55/97)	an ablation is interpretable and was run
STIX integrity pass	60 fresh judge bundles	reported by removal reason
outcome of the hint ablation	**mean Δ (on − off)**	**95% CI**
distinct technique identifiers	+0.50	[−3.33, +4.50]
claims	−0.83	[−4.33, +3.67]
seconds	+52.55	[−161.93, +268.45]

and cancelling. This is the fourth architectural claim to end in a null and the last one we were in a position to test.

That null is constrained by a second result: half the experiment was not scoreable. Twelve pairs were attempted and 6 survived: 3 lost to degenerate decode, an arm emitting 49 to 117 claims across 2 to 14 technique identifiers at ratios 8.4 to 34.3; 2 unattributable, both arms dead with the host state never recorded; and 1 incomplete at the 630 s bound, a genuine outcome that still costs the pair. A 50% pair-loss rate bounds what any ablation here can detect, and quoting +0.50 [−3.33, +4.50] without it would claim a precision the instrument does not have. The unattributable pairs are this paper's own retention rule recurring against it: the run halted at 10 of 24 arms and they died under host memory pressure with 2.3 GB of the model server's address space paged out, which Section 3.10 reports as a threat in its own right. We had retained per-sample outputs; what went unrecorded was the environment. Two further arms carried a `Connection error` from a restart race, meaning the measurement never happened, and were re-run, contributing a −4 against the hint; a third was the timeout, which is an outcome, and deleting an outcome one dislikes is selection rather than repair.

Running the ablation also surfaced M4 of Section 3.7, a production defect that 2,746 passing tests did not: on any binary rich enough to exhaust the 40-step ReAct budget the analyst returned zero techniques. Fixed and verified on the same sample, from 1,677 s to 323 s and from 0 to 5 technique identifiers, though the fix is partial: one sample still exceeds the bound, and generation varies from 162 to 20 tokens/s on this hybrid recurrent architecture, so a bounded input does not rescue a case where the tokens are eight times slower than assumed.

3.6. The reach of the corroboration cascade

The multi-layer corroboration set is the design's central claim about evidence quality. We remove one Layer-0 source at a time and compare the bundle the analyst receives against the one produced with all of them, under two conditions differing in whether the removed source solely owned its techniques. Under *disjoint*, where its techniques disappear because nothing else claims them, the verdict changes in 32 of 32 arms at Jaccard 0.750 [0.714, 0.800]. Under *overlap*, where the techniques survive under a partner source but their corroboration changes, it changes in 0 of 32 at Jaccard 1.000 [1.000, 1.000]. Removing a technique removes it from the bundle; destroying the cross-domain agreement behind it, which is what the cascade exists to compute, leaves the bundle byte-identical.

The reason is not the one we first wrote, and the correction is M5 of Section 3.7. The reconciliation step drops every `attack-pattern` whose technique identifier cannot be resolved and appends every cascade technique missing from the bundle, so the cascade's set is a guaranteed subset of every bundle the system emits whatever the judge produced. Recomputing each arm's cascade set from the same seeded fixtures shows the bundle is exactly that set in 80 of 80 arms, with the judge contributing zero techniques to any of them, so both conditions follow from set arithmetic rather than from restraint. The Jaccard value was the signal we missed, for the reason M5 gives.

That leaves two pipelines the evidence cannot tell apart: a judge whose output is unusable and wholly replaced, and one that reproduces the cascade's set exactly. Intercepting the reconciliation step and recording what the model produced before the deterministic set was merged in settles it (Table 9): three of the four calls produced nothing nameable, and the fourth named only techniques the cascade already held. The bundle carries the judge's name and the cascade's contents.

Table 9: What the judge contributed to the artefact the analyst receives, intercepted at the reconciliation seam. Counts are techniques, over four calls.

	total	per call
attack-patterns the judge emitted	50	12.5
carrying a resolvable ATT&CK identifier	12	3.0
dropped, the model named no technique	38 (76.0%)	9.5
identifiers the cascade did not already hold	0	0.0
injected because the judge omitted them	87	21.8

Half the calls never reached that step. Four of the eight timed out at the verdict ceiling, and on a timeout the verdict tool returns a text-fallback bundle that never calls the reconciliation routine, so the cascade is not consulted at all. The timeouts are not a property of the fixtures but of M7 (Section 3.7): the judge's 8,192-token output cap reached the local server under a name it does not read, so the model decoded unbounded until the caller gave up, one such call measured at 30,155 generated tokens and still going.

Repeating the study with the cap fixed turns the pair into a controlled experiment, and the result inverts this section's own conclusion. With the ceiling binding, the same four fixtures fail and the same four succeed, the judge's share of the bundle is 0.0% in both conditions, and every number on the reconciled path is identical; only the failure's arrival changes, 8,192 tokens of unparseable output in three and a half minutes instead of a ten-minute timeout. The artefact does not. The fallback builder scrapes ATT&CK identifiers from the evidence claims and from the model's own raw response, and in the uncapped condition that response is the literal string `[TIMEOUT]` and contains no identifiers, so the fallback bundle was exactly the evidence-derived set, identical to the cascade set on all four calls. In the capped condition it is 18,748 to 24,710 characters of degenerate decode (Table 10).

Table 10: Techniques reaching the analyst on the text-fallback path against those the corroboration cascade held, per fixture. The last column is the set no evidence source corroborated.

fixture	fallback	cascade	only in fallback
`jhuhugit`	32	20	**12**
`sardonic`	27	25	**2**
`sliver`	46	23	**23**
`wannacry`	26	16	**10**

47 techniques reach the analyst that the corroboration cascade never held, and none goes the other way; on one sample the bundle doubles. Because the uncapped run is a control whose raw text provably contains no identifiers, every one is attributable to the model's own output, and they passed through no filter at all: not the cascade, not the reconciliation step, not the invalid-identifier filter, not the integrity pass. Checked against the 691-technique catalogue they are also real, 45 of 45 unique identifiers existing and none invented, which removes the easy reading: these are not hallucinated identifiers a schema check would catch but genuine ATT&CK techniques that no evidence source claimed, in the same object type and the same bundle as techniques three independent sources agreed on, with nothing downstream to tell them apart. The judge therefore has no influence over which techniques reach the analyst on the path where it works and more on the path where it fails, 0 of 99 techniques when reconciliation runs against 47 of 131 when it does not: the architecture suppresses the model's contribution precisely when the model is functioning, and admits it precisely when it is not.

This replaces an earlier version that reported the verdict unchanged in 0 of 15 cases and carried two defects: the sources it varied excluded the Sigma layer, which fires on 94 of 97 samples at weight 0.55, above every static layer it did vary; and at five techniques over six sources the round-robin assignment left one source with nothing, so its arm matched the baseline by arithmetic. The re-run uses fixtures of 12 to 51 techniques so every source carries at least three claims, includes Sigma, and adds a corroborated pair that does not involve YARA; the null survives at 32 arms rather than 15. One pre-registered prediction resolved in both directions: removing the tool-artifact layer should change nothing, because it emits on YARA's cascade domain and cannot contribute corroboration, and under overlap that is right at 0 of 8 while under disjoint it is wrong at 8 of 8, because there the source solely owns its techniques. The prediction holds precisely where corroboration is the only thing varying.

3.7. Instrument failures

Seven failures of the instrument share a shape: it answered successfully, and answered about something else. Table 11 lists them with the layer each occurred at and what found it, and Figure 1 marks each on the topology. The first four crossed a boundary with another server. The last three are ours, two in the analysis code that produces our results and one in the pipeline that produces the artefact the analyst receives, so the shape stops neither at the network boundary nor at the evaluation harness.

Table 11: The seven instrument failures, with the layer each occurred at and what found it.

	what the instrument did	layer	what found it
M1	an optional argument the agent never supplied arrived as an explicit `null`, and a field typed `str` rejected it, failing all 36 tools	sandbox MCP client	every call returning the same validation error
M2	`load_program` answered `success` without changing the server's current program, so everything derived from it described the first binary of that container's lifetime	reverse-engineering server session state	call graphs identical to the character across unrelated samples
M3	a load the server could not perform answered HTTP 200 with an error in the body, so downstream code analysed whichever program remained current	HTTP status against body	66 consecutive samples at exactly 75,426 characters
M4	a 40-step ReAct budget and a wall-clock cap composed into a path that returned zero techniques, at 28 minutes per attempt	two local bounds	a 25-minute call always producing one claim and zero techniques
M5	an ablation varied a deterministic reconciliation step and the output was attributed to a language model	our analysis code	a Jaccard too clean for a model to produce
M6	a rank correlation over model size ranked a reasoning-configuration flag instead, recovering the published scaling coefficient	our analysis code	reading the arms table under the correlation
M7	a documented output ceiling was renamed during serialisation to a key the server does not read, so a component described as bounded was never bounded	our production pipeline	a study recording which failure branch each call took

M2 is the mechanism the others are read against: the response contained the correct program name. The server maintains a separate notion of the current program and `load_program` sets it only when nothing is current yet:

```
load A        -> {"success": true, "program": "A"}   current: A
load B        -> {"success": true, "program": "B"}   current: A   <-- still A
run_analysis  -> {"program": "A", "new_functions": 0}
call graph    -> A's graph, byte-identical, for every sample
```

From the second sample of a container's lifetime onwards, the decompilation, the imports, the strings, the call graph and the function hashes all described the first binary while the report named the current one. M1 and M3 are the same shape at neighbouring layers: M1 survived to first contact because the reverse-engineering server, the only one that client had ever talked to, declares no optional parameter with a typed default, and M3 is error swallowing in the ordinary sense, `raise_for_status()` seeing nothing wrong in a body reading `{"error": "Failed to load program from: ..."}`. M4 composed two bounds each correct alone, exceeding the step budget guaranteeing an attempt at the salvage and the salvage receiving a fresh copy of the full time budget.

A fifth case at the same boundary was caught before it produced data, and that difference is the argument of this paper. A request for a report the sandbox has since deleted is answered with HTTP 200 and a 63-byte body reading `{"error":true,"error_value":"Reports directory does not exist"}`, on fifty-six of one hundred tasks. A fetcher trusting the status code would have written those bodies into the archive under the filename of the sample they were meant to describe; it did not, because it verifies the report's own `target.file.sha256` against the identifier it asked for before writing, a four-line check that exists only because M3 taught us the shape. We first recorded the consequence as a retention limit, that the reports had existed and expired. That was wrong. Asking the sandbox how long each analysis had taken produced a split with nothing between the modes: the 43 tasks whose reports survived ran 186 to 366 s and the other 56 ran under a second, all 56 still marked `reported`. A Windows PE does not detonate in one second. Nothing had expired; nothing had happened, and the instrument said otherwise. Re-submitting two days later produced full-length analyses on the same instance, recovering 54 and putting the cohort at 97, while the remaining 3 produced no report on either attempt and none of them failed, all 3 still marked `reported` at zero seconds. A completion status is a statement about a queue rather than about an analysis, and it is exactly as trustworthy as an HTTP 200; the retrieval path now refuses any task under 60 s.

The last three crossed no boundary. M5 is the only one that reached a results table. We ablated the corroboration cascade and wrote the two conditions of Section 3.6 up as a finding about the judge, that it attends to the list of claimed techniques and ignores the evidence-quality apparatus above it. The judge was not involved: the reconciliation step makes the cascade's set a guaranteed subset of every bundle whatever the model produced, so both numbers are necessities of an `if` statement rather than measurements of restraint, and the experiment could not have returned anything else. A second study about to run would have compared a weighted cascade against a flat union of the same claims; both arms would have shared one reconciliation set, so it would have reported no difference correctly and vacuously, and it was stopped four minutes in by the same log line that exposed M5. The signal was in the results the whole time: a Jaccard of 1.000 with a zero-width bootstrap interval across 32 ablated arms at temperature 0.1 is not something a language model produces, it is what a constant looks like, and we read it as an unusually clean null because a clean null was the result we were prepared to find.

M6 is the only one whose wrong answer agreed with the literature. Assembling a parameter-size series to test the published finding that parameter count predicts ATT&CK-classification F1, the harness ranked mean F1 against total parameters across a 3.4× span and reported ρ=+0.866, reproducing the prior almost exactly, with the confounds it could not remove printed honestly beneath. The five rows were three models, one appearing twice because it had been run in two configurations, and that was not a labelling detail: the same weights score 0.3507 with the reasoning stream disabled and 0.0080 with it enabled, because 24 of 25 calls then spend their entire output budget reasoning and never answer, and the enabled row sat at the small end of the parameter axis where it set the sign. The failure is not the duplicate rows but that the arm-selection rule did not exist. The harness had averaged ranks, an exact permutation p, and a common-cell restriction so endpoint availability cannot masquerade as model size; every guard was about a way the numbers could mislead and none about whether the rows were comparable in the first place. The rule now keys on the measured reasoning share of each arm rather than on the flag requested. M5 was caught by a number too clean to believe; M6 produced one in exactly the range a reader would expect, in the direction the literature predicts, so nothing was anomalous to notice and it was caught by asking why one model was on the arms table twice.

M7 is in the production pipeline rather than in an evaluation harness, and it removed a guarantee the code states in its own comment. The judge is built with an 8,192-token output ceiling, and the line that builds it says why: to bound the verdict generation so a degenerate decode cannot consume the full wall-clock timeout. The client library renames that parameter to the vendor's newer spelling when it serialises the request, and the local inference server does not read the newer spelling. It accepted the field, ignored it, and decoded without a ceiling: measured on one call, a 1,403-token prompt, 30,155 tokens generated, still generating at 46 tokens per second when the caller's ten-minute wrapper gave up. The consequence is not a slow call but the four unreturned verdicts of Section 3.6, each of which left the pipeline on its text-fallback path. It was invisible because the configured value was correct at every level anything reads: the container passed it, the model object held it, and it survived every later rebinding. Only the serialised request was wrong, and nothing in the system reads the serialised request, so every check anyone would think to perform would have confirmed the property that did not hold. What found it was a study measuring something else, which recorded which branch each failed call took rather than only that it failed; four calls named the timeout branch, which prompted the question of why a temperature-zero model would need ten minutes.

The incompatibility itself is known, reported against `llama.cpp` and `vLLM` in their own issue trackers [53, 54] [preprint]; what one costs inside a measurement is the part we report.

That 2,746 tests passed throughout is a fact about test shape rather than test quality. M2 and M3 require a second case within one server lifetime, and a unit test loads one program, asserts, and tears down; M1 required a second server, and the behaviour was correct against the only one exercised; M4 required an input rich enough to exhaust the step budget, which small fixtures never are. Preconditions of that kind are exactly what a fast unit suite is designed to avoid. The last three are invisible to unit testing in principle, because every unit behaved as specified: M5's reconciliation function was never wrong and no component test can catch a misattribution made three modules away; M6's guards every one concerned the wrong question; and a unit test of M7 would check that the output ceiling is 8,192, which it is at every level the object model exposes.

3.8. Output cardinality as a detector

Five of the seven were found by the same observation, a number that repeated where variation was expected (Table 12). M6 and M7 were not, because their outputs vary and there is no repetition to notice.

Table 12: What the output-cardinality check saw and which defect each observation turned out to be.

observation	defect
a priority hint of exactly 2,575 characters for two unrelated binaries, and a third in an earlier session	M2
call graphs identical to the character (404,337 chars, 11,798 lines) for samples of 241 KB and 139 KB	M2
66 consecutive samples at exactly 75,426 characters	M3
every tool call returning the same validation error	M1
a 25-minute call that always produced one claim and zero techniques	M4
a Jaccard of 1.000 with a zero-width interval across 32 ablated arms at temperature 0.1	M5

The generalisation is one line of arithmetic. Before a batch measurement is trusted, the number of distinct outputs the N inputs produced is counted; if far fewer than N differ on a dimension that should vary, such as output length, digest or element count, the instrument is repeating itself, and repetition is what stale state looks like from outside. It needs no ground truth and no oracle, and it is reported as a result column rather than run as a test because the signal exists only in the batch. M5 extends it to our own results, where the repeated number was ours and the variation that failed to appear was the model's.

Plotted against inputs processed, distinct outputs track the diagonal for a healthy instrument and go flat for a stuck one, so the diagnosis is in the shape. Panel a) of Figure 5 is measured, its staircase's flat treads being genuinely identical binaries

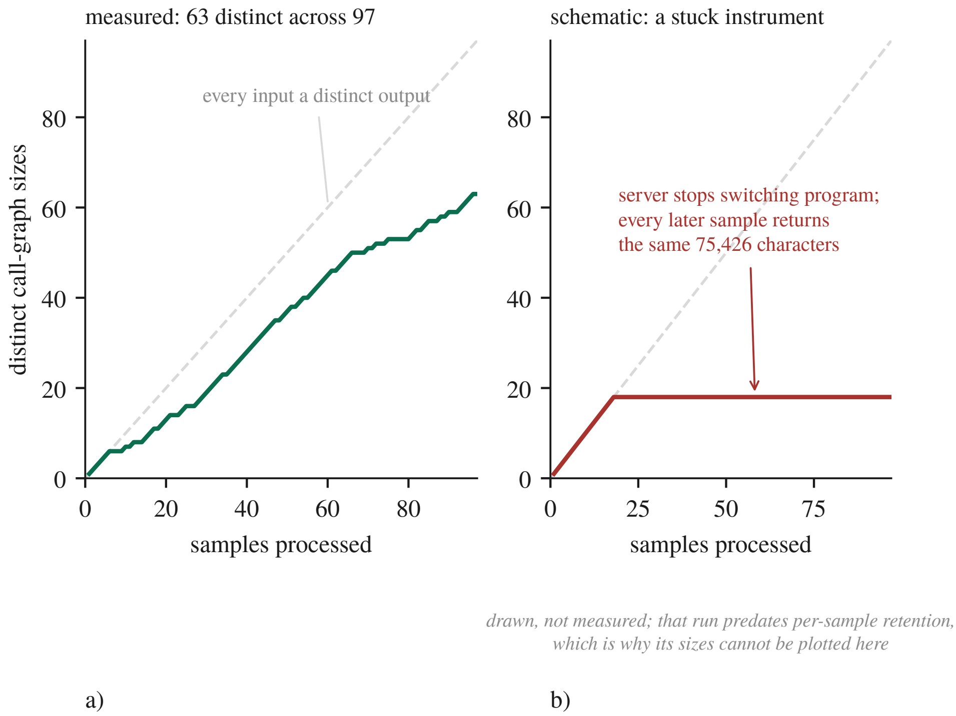


Figure 5: The whole detector, drawn. a) a measured run at 63 distinct call-graph sizes across 97 samples, b) a schematic of a stuck instrument.

rather than a fault. Panel b) is a schematic, and the reason is itself the cost of these failures: it depicts M3, the run in which 66 consecutive samples returned a byte-identical 75,426-character call graph, and we cannot plot it because that run predates the per-sample retention policy those very failures caused us to adopt.

The price is a completed study. A seven-year 210-sample temporal-drift analysis ran the full pipeline per sample against a long-lived server never restarted between samples, which is M2's precondition exactly, and whether it was affected cannot be determined because its per-sample outputs were not retained. The withdrawal is caused by the defect plus the inability to re-ask the question afterwards.

3.9. Discussion

The most consequential property of these negative results is that most of them are bounds rather than equivalences, and that was not evident when they were first written down. Five of the studies here run on five fixtures, where the sign-flip floor of Equation (2) puts α=0.05 out of reach whatever the effect size, and no number of extra decoding repeats moves it because repeats add rows inside clusters rather than clusters. The practical consequence is that a null has to be reported with its resolution: the frontier comparison could detect 0.300 F1 at 80% power and returned +0.0026, and only the pair of those facts is informative, since the second alone reads as equivalence and the first alone as a failed experiment. It also changes what the cheap fix is. The instinct on seeing an underpowered paired design is to add repeats, because repeats are cheap and samples are expensive, and on clustered data that instinct is exactly wrong: a sixth fixture would have halved the attainable p, while a sixth repeat on five fixtures moves it not at all.

The literature we counter-searched against locates silent failures at the boundary between modules, and four of our seven crossed such a boundary while three crossed none. What the seven share is not a boundary but an attribution. An ablation varied the deterministic reconciliation step and we attributed the difference to a language model; a rank correlation ordered a reasoning-configuration flag and we attributed it to parameter count; a renamed output ceiling let us attribute boundedness to a component that had never once been bounded. In every case the number was real and the arrow pointing to its cause was drawn by us. We offer that as a proposal rather than as a taxonomy, since seven cases from one project is not a class, but it does suggest a different question to ask of an evaluation than boundary-oriented thinking suggests: not whether any component failed silently, but, for each number about to be reported, whether its cause was measured or assigned. The clustered-inference correction of Section 2 is that question turned on ourselves and answered badly, since every interval in an earlier draft attributed its width to sampling variation among independent observations, the observations were not independent, and nothing in the pipeline, the tests or the review caught it because the defect was never in a function.

What a practitioner should take from this is narrow, because it is one pipeline on one machine. The firing rate should be measured before the effect, since a mechanism engaging on 0.82% of its inputs produces an ablation whose null describes the cases where it never ran; the distinct outputs should be counted, because that needs no ground truth and belongs beside the sample size rather than inside a test; and per-sample outputs should be retained, along with whatever else the study is made of. We adopted retention after losing a study to its absence, scoped the rule to the object under study, and the next failure was in the environment, where four arms died on host memory and we had their outputs but not the host state. A rule written after a failure is scoped to that failure.

What we are not in a position to say needs the same precision. Negotiation is not shown to be worthless in general, because a control matching the treatment locates our own gain and not anyone else's. Model capacity is not shown to be off the ceiling, only that a larger model did not separate from ours at this design's resolution while differing in a configuration flag worth more than any architecture we measured. Verbal confidence is not shown to be uninformative in general, only indistinguishable from chance here. Nor are our seven mechanisms shown to generalise. What we can say is what each of them cost.

3.10. Limitations and threats to validity

This section is organised around the nine pitfalls of *Chasing Shadows* [37], with two checks added that the survey does not have and that both caught errors here.

It begins with damage rather than with hazards: one study is withdrawn, the 210-sample temporal-drift analysis of Section 3.8, which was completed and had entered our claim ledger as a novel result, and it is not restated anywhere in this paper. Table 13 gives our exposure to each pitfall with the basis for the verdict.

Table 13: Our exposure to each of the nine pitfalls, with the basis for the verdict.

pitfall	verdict	basis
P1 data poisoning	`CLEAR`	nothing is trained; retrieval corpora are curated and their provenance documented
P2 label inaccuracy	`CLEAR`	ground truth is MITRE-curated and no LLM scored any result; no human analyst rated any report either
P3 data leakage	`PARTIAL`	one instance found by us, disclosed and mitigated, but never systematically audited; the model's training data is unknown to us and memorisation was not probed
P4 model collapse	`CLEAR`	augmenting the long-term memory corpus with LLM-generated cases was considered and refused
P5 spurious correlations	`PARTIAL`	two were found and are reported; no systematic perturbation testing was performed
P6 context truncation	`EXPOSED`	truncation is designed in throughout and the counters now exist, but the distribution over a full cohort does not
P7 prompt sensitivity	`PARTIAL`	a structured prompt-variation study exists, but production prompts are fixed and were not varied per model
P8 surrogate fallacy	`PARTIAL`	every local result comes from one model on one machine, and a second endpoint has now run to completion against it
P9 model ambiguity	`PARTIAL`	model and engine are pinned by digest, revision and commit, but the commit was reconstructed rather than recorded

Three of the nine need more than a row. On P6, truncation enters through chunking at function boundaries, a per-call evidence cap, a 40-step ReAct budget, an 8,192-token verdict cap and a 400-char hint cap, and that enumeration was itself incomplete: a 20,000-edge cap on the call-graph fetch was discovered only while instrumenting a different measurement and silently truncates 1 of 97 samples. Bounds also compose, as M4 shows, and removing a hint made the judge overrun a 600 s ceiling and return an empty bundle 6/17 times.

P8 is the pitfall this work is most exposed to and the one where the evidence has moved. A 120B reasoning model has now run to completion against the local model and did not separate from it (Section 3.4); a 3.4× parameter advantage producing no measurable difference is evidence against the pitfall's usual worry rather than merely an absence of evidence for it. That null is confounded by the reasoning flag, which cannot be removed on that endpoint and is worth 0.34 to 0.45 F1 on two separate models. Quantisation is no longer unmeasured under P8 either (Section 3.4), though two serving stacks differ alongside the precision, so the bound is on the deployment rather than on quantisation alone. What P8 cannot close is the size series, for a measured rather than a budgetary reason: testing a parameter-size trend needs three arms at three sizes matched on the flag that outweighs size, the one endpoint honouring the flag hosts the 35B model we already run, and the one above 35B does not. Coverage also remains, the second model having been measured on fixtures rather than on the malware cohort. P9 is `PARTIAL` because the running binary reports its version as `unknown` and the engine commit was reconstructed rather than recorded (Section 2.4); build provenance must be captured at build time, and ours was not.

Two checks the survey does not include, both described in Section 2, caught errors one level below what the nine pitfalls address. A scored component deriving its output from the same source as the labels is scoring a tautology, so this was tested component by component rather than assumed. Ground truth is the MITRE `malware uses attack-pattern` relationship set, so it comes from nothing this project produces. The Sigma layer reads its identifier from each rule's own `attack.tNNNN` tag, and those rules are community-authored, so Sigma shares ATT&CK's vocabulary with the labels and not their source. The sandbox baseline, which would have mattered most, takes its identifiers from class attributes hand-written in each signature module and consults no family attribution anywhere, which is what makes it a valid baseline rather than a second view of the answer. And the retrieval corpora do not overlap the evaluation cohorts, checked by digest: of the 1,733 cases in the ATT&CK case corpus, none shares a SHA-256 digest with the 100-sample dynamic cohort or the 210-sample drift manifest. Two construct problems survive and both are ours. The case corpus is labelled with the technique identifiers the pipeline itself previously attributed, which is why that number is relabelled as self-consistency in Section 3.5; and the catalogue defining valid technique identifiers is the same file that bounds the label universe.

Any claim about what the sandbox contributes has to survive one measurement of what the sandbox reports. Across the 97 archived analyses there are 143 distinct network domains and 38 of them appear in every single sample, the analysis virtual machine's own vendor telemetry, present whatever was detonated. Per sample, between 55.9% and 79.2% of observed domains are cohort-ubiquitous, median 67.9%, so two-thirds of the network evidence in a dynamic arm is the instrument describing itself and an effect attributed to malware behaviour may be partly an effect of that machine's idle traffic. The same measurement bounds two Layer-0 sources from the other direction: the living-off-the-land binary and domain generation algorithm layers produced a claim on none of the 95 reports archived when that ablation ran, while being fed a median of 8,667 recorded API calls and 48-–68 domains respectively, so neither appears as an arm in the cascade ablation. An earlier version of this measurement on the 43 analyses that survived before the cohort was recovered gave 130 domains with 40 ubiquitous at a median share of 71.4%: more than doubling the cohort moved the figure by three points and left the conclusion where it was.

The machine a measurement runs on is part of the instrument, which is a threat we did not anticipate and met late. The working set does not fit a 31 GiB host alongside an interactive session, with the model server holding ~16 GB, the analysis worker ~8 GB and the disassembly container up to 6 GB. During a paired ablation the swap file was exhausted and the model server had 2.3 GB of its own address space paged out; an arm then exceeded a 594-second budget on a prompt trimmed to 16,000 characters. Whether that arm failed because of the pipeline or because of the host cannot be determined, because we recorded the pipeline's outputs and not the host's state, so the affected arms are reported as `unattributable` and the ablation as incomplete rather than as a null. Wall-clock bounds are therefore host-dependent measurements presented as system properties, so any result here that turns on a timeout is a statement about this pipeline on this machine under whatever load it had; and the screen we added can only be applied forward, since arms already collected without host state cannot be re-screened, which is the same structural problem that withdrew the drift study appearing in a new variable.

Three objections remain that the work does not close. The first is that the judge findings rest on four calls, the intercept requiring a live judge call per observation. On the working path the finding is deductive rather than statistical, since the reconciliation step restores the cascade's set by construction and the 80-arm mechanism check confirms the bundle equals the cascade set on every arm, the judge contributing one of its own on 0 of 8 samples, an upper bound of 0.375 on the per-sample rate. On the failing path it is an existence claim, demonstrated on four calls against an uncapped control whose raw text provably contains no identifiers. Neither is a rate, and what would settle it is the same intercept over the 97-sample cohort. The second is that the detector's own evidence is retrospective: it found four defects on batches we already had reason to distrust, and the prospective test, reporting output cardinality beside every batch for a year and counting how often it is first to a defect, is the experiment and we have not run it. The third is that seven defects in one's own code is a post-mortem rather than a result. We claim the setting and the price rather than the category, every mechanism having been counter-searched in an adjacent field's vocabulary by a search that demoted more than it confirmed.

All three are blocked by the same thing, a host on which the model server does not fit alongside anything else, so a long run walks the host into its own memory guard. Cycling the server from the checkpoint between stretches of work is what closed a fourth objection, that the equal-budget arms ran only on 5 synthesised fixtures whose evidence is in bijection with their answer key: two of the three were re-run on real binaries and the answer changed sign (Section 3.3). The frontier comparison and the confidence calibration still rest on that corpus and are reported as a pilot.

4. Conclusion

This work addressed two barriers to trusting a measurement of an LLM malware-analysis pipeline: that such pipelines are rarely scored against the deterministic tooling they are built on, and that the instrument producing the score is itself a distributed system whose failures do not announce themselves. We built such a pipeline and then spent longer measuring it than building it, at equal token budget against simpler alternatives, with a noise control and with intervals resampled at the level the observations are independent at.

Conceptually, the contribution is a way of reading what a component adds that does not depend on the component working: a baseline to refer an F1 to, a firing rate to decide in advance whether an ablation can say anything, a minimum detectable effect to turn a null into a bound, and a count of distinct outputs to ask whether the instrument produced N answers or one answer N times. None is expensive and each changed a conclusion here.

Four architectural claims motivated the design and one of them survives in part. Where a corpus large enough to produce a significant answer was available, decomposing the analysis into several model calls is worth +0.0537 F1 over a single judge and the negotiation between them is worth +0.0005: the design helps, and not for the reason it was built. The other three do not survive. The corroboration cascade never reaches the artefact the analyst receives, two-tier attribution fires too rarely to carry the result, and the confidence number every deterministic gate consumes is indistinguishable from chance. What does move the end-to-end result is the layer underneath: the deterministic assigner gives the cleanest identifier gate of three backends on two external corpora, and when its reconciliation runs the judge contributes no technique the cascade did not already hold, while on the path that bypasses it uncorroborated identifiers reach the analyst. The finding we did not expect to be the paper is that the instrument was repeatedly wrong in ways that looked like data. Seven mechanisms are catalogued with the layer each occurred at, each returned a plausible result rather than an error, and 2,746 passing tests caught none of them. Four crossed a boundary with another server and were found by one arithmetic check on how many distinct outputs the inputs produced; in the three that crossed none, a measurement's cause was assigned rather than measured.

We are careful about what is new. Silent failure at tool boundaries is documented, the metamorphic relation behind the detector is ordinary, and inference-time configuration inverting model rankings inside a constrained budget is established. What we add is the setting and the price: one completed study withdrawn because its per-sample outputs were not retained, then four arms of a later ablation lost the same way one level up, because the rule written after the first failure was scoped to that failure and the environment a study runs in is also part of the instrument.

The limitations bound the reading directly. This is one pipeline, one 35B model at one quantisation, on one machine, evaluated on Windows PE malware against family-level ground truth, and most of its nulls are bounds set by a five-cluster corpus rather than equivalences. Future work follows from each: re-running the withdrawn drift study under the corrected instrument, extending the judge intercept from four calls to the full cohort, running the parameter-size series on endpoints that honour the reasoning flag, and testing the output-cardinality check prospectively rather than retrospectively. Each needs a host the model server does not fill on its own, which is the binding constraint on this work rather than the method.

What we do assert is a change of default. In a pipeline assembled from other people's servers the correctness of the model is not the binding constraint on the correctness of the result. Long before a benchmark score is a claim about a system, it is a claim about an instrument, and a server that answers is not the same as a server that answered the question it was asked.

Declarations

CRediT authorship contribution statement

Düzgün Küçük: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing (original draft). *Fatih Ertam*: Conceptualization, Supervision, Project administration, Writing (review and editing).

Ethics, containment and data sources

Every sample analysed in this work is live Windows PE malware. The 97-sample cohort and the 210 dated samples of the withdrawn drift study were obtained from MalwareBazaar [56], under terms that permit research redistribution of samples but not of the corpus as a package. MABEL, the Malware Analysis Benchmark for AI/ML, was mined offline into the family-fingerprint catalogue and the ATT&CK case corpus with no binaries downloaded, and Ultimate-RAT-Collection supplied the remote access trojan builders and payload binaries for that catalogue and for the leakage-free retrieval split. The ATT&CK Enterprise knowledge base [57] supplies the family-to-technique `uses` relationships used as per-sample ground truth, under the ATT&CK Terms of Use.

Detonation happens only inside a CAPE sandbox on a dedicated Windows guest image reverted to a clean snapshot between analyses. The sandbox host is reachable from one network segment and from no public route, holds no credentials for any other system, and shares no filesystem with the host that runs the pipeline. Binaries are held only in a directory excluded from the host's real-time scanner, are never committed to version control, and are not redistributed with this paper. We note a limitation of that arrangement rather than claiming it sufficient: the guest reaches the public internet during detonation, because a sample that cannot resolve its command-and-control infrastructure behaves differently from one that can and the network evidence channel would otherwise be empty for every sample. Live detonation with egress is the standard arrangement for behavioural sandboxing, and it is the arrangement whose consequences we report. No user study was conducted, no personal data was collected, and no human analyst scored any output, that last point being a limitation of the evaluation rather than a claim about ethics.

Data availability and reproducibility

The complete project is released under an open licence as an archived software release [58]: the pipeline itself, the evaluation harnesses, the derivation of every number in this paper, the per-sample records they read and the scripts that build the figures and assemble the document. That release is what the statements below refer to, rather than the development repository, because a repository moves and a claim about an artefact should name something that does not. The archive holds every per-sample record retained by every study reported here, in the JavaScript Object Notation files the harness wrote, together with the offline re-analysis that recomputes every interval, design effect, minimum detectable effect and adjusted p-value from those records and its random seed, the ATT&CK ground-truth fixtures, and the model digest and engine commit needed to reproduce the local arm. Two runs of the offline analysis produce byte-identical output, asserted by a test in the released suite rather than claimed here, and every interval carries the seed, the iteration count and the number of clusters it was computed from. The live arms are not bit-reproducible and we do not claim they are: decoding is at temperature 0.1 on the local arm and at each provider's default on the hosted ones, one of which accepts a reasoning-configuration parameter and does not act on it, so anyone re-running them should expect the numbers to move and should read the intervals accordingly.

Four things are withheld. Malware binaries are not released and are obtainable from the sources named above by the SHA-256 digests in the archive. Sandbox reports are not released in full because they embed strings and memory contents from the samples. The sandbox host's address appears nowhere in the archive. And one dataset used for retrieval-corpus construction is redistributed by its authors under terms that do not permit our redistribution, so it is referenced rather than included. One thing is not available at all: the 210-sample temporal-drift study's per-sample outputs do not exist, having been written to a path that was valid on the machine the harness was first written on and silently relative on the machine it ran on. The study is withdrawn on that account and the withdrawal is reported in the results rather than quietly repaired. The manifest of the 210 samples survives and is included, so the cohort can be reconstructed even though the results cannot.

Use of generative artificial intelligence

Large language models are the object of study in this work, so the pipeline under evaluation is built from them and every measurement reported here is a measurement of their behaviour in it. Separately, generative artificial intelligence was used as an authoring and engineering aid, for drafting and revising prose, writing evaluation harnesses and their tests, and performing literature searches whose results were then verified by fetching each source. Of the nine citations added by those searches and checked one by one, six said something the source did not, and all six are corrected in the text, because a search summary that reads as a citation is the same class of failure this paper is about. An internal readability assessment of report narratives was performed with a language model as a development aid; its results are deliberately excluded and no LLM scored any result reported here. The authors take responsibility for all content, including every number, every citation, and every claim about what the measurements support.

Declaration of competing interest

The authors declare no competing financial or non-financial interests, and no external funding supported this work. The three commercially hosted inference endpoints were paid for at their public list prices, and their providers had no involvement in, or advance knowledge of, this evaluation.

Appendix A. The prompts and the output contract

This paper measures what a model adds to a pipeline, and that measurement cannot be read without knowing what the model was asked. The full text of the four prompts is in the released repository [58]; what is reproduced here is the part the paper's own claims rest on, because a claim about falsification before confidence is a claim about specific sentences in a specific prompt.

Appendix A.1. What the three analyst prompts share

Each analyst receives one system prompt with the same five parts, and the parts are the same length in each: a role and an evidence rule, a tool workflow, a set of conditional tools, a verification discipline, and a set of standing cautions. The evidence rule is identical across the three and is the pipeline's grounding constraint: for every claim the model makes it must cite a concrete artefact, which for the static analyst is a function name, a string offset, an import or a hex pattern, for the dynamic analyst an API call or a registry write from the sandbox report, and for the network analyst a flow or a resolved domain. What differs between the three is the tool server each is bound to, the evidence type it reads, and the technique families its role paragraph names.

Appendix A.2. The confidence rules, as shipped

Two clauses of the verification discipline carry the fourth architectural claim of Section 1.2 and set the cap whose firing rate Section 3.5 measures at 0.82% of techniques. They are quoted from the static analyst's prompt without alteration:

```
- A SPECIFIC claim (a named algorithm like RC4/djb2/ROR13, a constant
  or XOR key, or a hash-resolved API) may reach CONFIDENCE >= 0.8 only
  if you FALSIFY it first: `emulate_function` with a known input vs the
  expected output, OR `analyze_dataflow(direction=backward)` to confirm
  its origin. If you cannot run the check (non-leaf, syscall/heap side
  effects), cap CONFIDENCE at 0.7.

- A claim is High (>= 0.8) only with >= 2 independent evidence loci
  (e.g. an import AND its call-site). A single locus caps at 0.7.
  Reconcile any contradictory signals before emitting.
```

Two further clauses of the same block are elided rather than silently dropped: one governs disambiguation when a hash-resolution tool returns colliding candidates, and one refuses a particular obfuscation technique on the evidence of dynamic import resolution alone. Neither bears on a measurement reported here, and both are in the release.

The rules are worth reading against the result. They ask the model to earn a number above 0.8 by running a check, and to cap itself at 0.7 when it cannot. The number they produce is the one every deterministic gate downstream consumes, and Section 3.5 finds it near chance, with 186 of 210 claims arriving at exactly 1.0. Whether the discipline is not followed, or is followed and does not separate correct claims from wrong ones, this design cannot distinguish.

Appendix A.3. The output contract

An analyst answers in blocks, and a block is the unit everything downstream counts. The parser requires the first field and defaults the rest, because a block that names a finding and omits its confidence is still a finding:

```
CLAIM: <the finding>
EVIDENCE: <a concrete artefact reference>
CONFIDENCE: <0.0 to 1.0>
TECHNIQUE: <T1055.001 | NONE>
---
```

Each parsed block becomes one record, and its four fields are what the rest of the system sees. The claim and the evidence reference are free text. The confidence is bounded to the unit interval by the schema rather than by the prompt, which is why a model that ignores the verification discipline still produces a well-formed number. The technique identifier is optional and is pattern-checked where it is declared:

```
technique_id: str | None = Field(None, pattern=r"^T\d{4}(\.\d{3})?$")
```

A malformed identifier is therefore refused at the schema and never reaches the cascade. This is the filter that Section 3.6 finds the text-fallback path bypassing, which is why the identifiers admitted there are real ones no evidence source claimed rather than invented strings.

Appendix A.4. What varies per sample

The system prompts are fixed strings and are not templated per sample, which is the reason Section 3.10 records prompt sensitivity as `PARTIAL`: production prompts were held constant and were not varied per model. What varies is the human turn, which carries the sample's path for the static analyst, the sandbox report for the dynamic analyst and the network observations for the network analyst, each wrapped in delimiters that mark it as untrusted input. Evidence is truncated to a per-call character cap before it is interpolated, and that cap is one of the truncation points enumerated under P6.

References

[1] M. Büchel, T. Paladini, S. Longari, M. Carminati, S. Zanero, et al., SoK: Automated TTP extraction from CTI reports, are we there yet?, in: 34th USENIX Security Symposium, 2025, pp. 4621–4641.

[2] H. S. Gunawi, C. Rubio-González, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, B. Liblit, EIO: Error handling is occasionally correct, in: 6th USENIX Conference on File and Storage Technologies (FAST), 2008, measures error loss at module boundaries: 1153 of 9022 propagation channels (13%) drop an error code without handling it, and inter-module calls are a major cause.

[3] D. Yuan, Y. Luo, X. Zhuang, G. R. Rodrigues, X. Zhao, Y. Zhang, P. U. Jain, M. Stumm, Simple testing can prevent most critical failures: An analysis of production failures in distributed data-intensive systems, in: 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2014, an error handler that only logs the error is counted as ignoring it; 25% of catastrophic failures arise that way, and 92% from incorrect handling of errors that were explicitly signalled.

[4] A. Alquraan, H. Takruri, M. Alfatafta, S. Al-Kiswany, An analysis of network-partitioning failures in cloud systems, in: 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2018, pp. 51–68, "The majority of the failures were silent", and where a warning was returned "all returned warnings were confusing, with no clear mechanism for resolution".

[5] J. Xue, Y. Zhao, Y. Wang, J. Chen, D. Wang, A characterization study of bugs in LLM agent workflow orchestration frameworks, in: 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), Industry Showcase, 2025, pp. 3369–3380, 1026 bugs across LangChain, LlamaIndex and Haystack; nine root causes and six symptom categories. `doi:10.1109/ASE63991.2025.00278`.

[6] M. Liu, S. Zhong, W. Bi, Y. Zhang, Z. Chen, Z. Chen, X. Liu, Y. Ma, A first look at bugs in LLM inference engines, ACM Transactions on Software Engineering and Methodology929 bugs across llama.cpp, vLLM and four other engines; the taxonomy carries both "Incorrect Request Parsing" as a root cause and "Silent Error" as a symptom (2026). `doi:10.1145/3788873`.

[7] W. Yu, et al., Maltracker: A fine-grained NPM malware tracker copiloted by LLM-enhanced dataset, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2024. `doi:10.1145/3650212.3680397`.

[8] W. Zhao, et al., AppPoet: LLM-based Android malware detection via multi-view prompt engineering, Expert Systems with Applications 262, arXiv:2404.18816 (2025). doi:10.1016/j.eswa.2024.125546.

[9] A. Rollinson, N. Polatidis, LLM-generated samples for Android malware detection, Digital 6 (1) (2026) 5. doi:10.3390/digital6010005.

[10] S. Saha, et al., MaLAware: Automating the comprehension of malicious software behaviours using LLMs, in: IEEE/ACM International Conference on Mining Software Repositories (MSR), 2025, arXiv:2504.01145. doi:10.1109/msr66628.2025.00037.

[11] Center for Threat-Informed Defense, TRAM: Threat report ATT&CK mapper, Open-source software, MITRE Engenuity, not a paper and cited as the artefact it is; the repository names no preferred citation (2023).

[12] V. Legoy, M. Caselli, C. Seifert, A. Peter, Automated retrieval of ATT&CK tactics and techniques for cyber threat reports, in: 1st Cyber Threat Intelligence Symposium (CTI), 2020, arXiv:2004.14322. Recorded as a conference contribution by the University of Twente research portal; not indexed under this title in dblp.

[13] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, X. Niu, TTPDrill: Automatic and accurate extraction of threat actions from unstructured text of CTI sources, in: Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC), 2017, pp. 103–115. doi:10.1145/3134600.3134646.

[14] K. Satvat, R. Gjomemo, V. N. Venkatakrishnan, EXTRACTOR: Extracting attack behavior from threat reports, in: IEEE European Symposium on Security and Privacy (EuroS&P), 2021, pp. 598–615. doi:10.1109/eurosp51992.2021.00046.

[15] Z. Li, J. Zeng, Y. Chen, Z. Liang, AttacKG: Constructing technique knowledge graph from cyber threat intelligence reports, in: Computer Security, ESORICS, 2022, pp. 589–609. doi:10.1007/978-3-031-17140-6_29.

[16] M. T. Alam, D. Bhusal, Y. Park, N. Rastogi, Looking beyond IoCs: Automatically extracting attack patterns from external CTI, in: 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID), 2023, pp. 92–108, the LADDER framework. doi:10.1145/3607199.3607208.

[17] A. Lekssays, U. Shukla, H. T. Sencar, M. Parvez, TechniqueRAG: Retrieval augmented generation for adversarial technique annotation in CTI text, in: Findings of the Association for Computational Linguistics (ACL), 2025, arXiv:2505.11988. doi:10.18653/v1/2025.findings-acl.1076.

[18] F. Morbiato, M. Keller, P. Nair, L. Romano, Hierarchical retrieval augmented generation for adversarial technique annotation in cyber threat intelligence text, PREPRINT, not peer reviewed. arXiv:2604.14166 (2026). doi:10.48550/arXiv.2604.14166.

[19] Y. Cheng, C. Li, R. S. P. Basuki, Q. Cui, W. Ding, P. Gao, TTPrint: Evidence-grounded TTP extraction via diverge-then-converge verification, PREPRINT, not peer reviewed. arXiv:2605.25836; the Comments field says so in as many words. Kept because its evidence-grounding requirement is the design constraint our own gate obeys, so dropping it would let us appear to have set that bar ourselves (2026). doi:10.48550/arXiv.2605.25836.

[20] K. D'Oosterlinck, O. Khattab, F. Remy, T. Demeester, C. Develder, C. Potts, In-context learning for extreme multi-label classification, PREPRINT, not peer reviewed. arXiv:2401.12178, v1 only, no Comments and no journal reference (2024). doi:10.48550/arXiv.2401.12178.

[21] A. Abdallah, J. Holdcroft, M. Ali, A. Jatowt, Are LLM-based retrievers worth their cost? an empirical study of efficiency, robustness, and reasoning overhead, in: Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2026, arXiv:2604.03676. doi:10.1145/3805712.3808566.

[22] L. Lange, H. Adel, et al., AnnoCTR: A dataset for detecting and linking entities, tactics, and techniques in cyber threat reports, in: Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING), 2024, arXiv:2404.07765. CC-BY-SA 4.0. doi:10.63317/4esp6coivous.

[23] H. T. Ng, et al., The CoNLL-2014 shared task on grammatical error correction, in: Proceedings of the Eighteenth Conference on Computational Natural Language Learning: Shared Task, 2014, cited with the BEA-2019 shared task for the $F_{0.5}$ convention, which weights precision twice recall to penalise over-correction. The convention is documented by the shared tasks themselves. doi:10.3115/v1/w14-1701.

[24] C. Bryant, M. Felice, Ø. E. Andersen, T. Briscoe, The BEA-2019 shared task on grammatical error correction, in: Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications, 2019. doi:10.18653/v1/w19-4406.

[25] S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, J. Weston, Neural text generation with unlikelihood training, in: International Conference on Learning Representations (ICLR), 2020, arXiv:1908.04319. Welleck et al. blame the likelihood objective itself; the training-data account is the other paper's.

[26] H. Li, T. Lan, Z. Fu, D. Cai, L. Liu, N. Collier, T. Watanabe, Y. Su, Repetition in repetition out: Towards understanding neural text degeneration from the data perspective, in: Advances in Neural Information Processing Systems (NeurIPS), 2023, arXiv:2310.10226. Argues that self-reinforcement explanations reduce to the training-data account. `doi:10.52202/075280-3187`.

[27] G. Chen, H. Sun, D. Liu, Z. Wang, Q. Wang, B. Yin, L. Liu, L. Ying, Re-Copilot: Reverse engineering copilot in binary analysis, PREPRINT, not peer reviewed. arXiv:2505.16366, v1 only, no Comments and no journal reference. Cited only as one of the open-weight binary specialists that establish the model-tier split (2025). `doi:10.48550/arXiv.2505.16366`.

[28] H. Tan, Q. Luo, J. Li, Y. Zhang, LLM4Decompile: Decompiling binary code with large language models, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2024, pp. 3473–3487. `doi:10.18653/v1/2024.emnlp-main.203`.

[29] N. Jiang, C. Wang, K. Liu, X. Xu, L. Tan, X. Zhang, P. Babkin, Nova: Generative language models for assembly code with hierarchical attention and contrastive learning, in: International Conference on Learning Representations (ICLR), 2025, poster. arXiv:2311.13721.

[30] Z. Xuan, X. Xu, M. Zheng, L. Z.-H. Tan, J. Guo, T. Zhang, L. Yu, C. Wang, X. Zhang, Identifying adversary tactics and techniques in malware binaries with an LLM agent, PREPRINT, not peer reviewed. TTPDetect. arXiv:2602.06325. Kept because it is the nearest system to ours and the sentence citing it concedes ground rather than claims it; dropping it for want of a venue would let this paper imply it holds that ground itself. No peer-reviewed system with the same scope was found (2026). `doi:10.48550/arXiv.2602.06325`.

[31] K. Kate, T. Pedapati, K. Basu, Y. Rizk, V. Chenthamarakshan, S. Chaudhury, M. Agarwal, I. Abdelaziz, LongFuncEval: Measuring the effectiveness of long context models for function calling, PREPRINT, not peer reviewed. arXiv:2505.10570, v1 only, no Comments and no journal reference. Kept because it says the tool-catalogue degradation we report is already documented in general, which is the direction that costs us (2025). `doi:10.48550/arXiv.2505.10570`.

[32] V. Paramanayakam, A. Karatzas, I. Anagnostopoulos, D. Stamoulis, Less is more: Optimizing function calling for LLM execution on edge devices, in: Design, Automation and Test in Europe Conference (DATE), 2025, arXiv:2411.15399. Both quoted figures were confirmed against the full text. `doi:10.23919/date64628.2025.10992798`.

[33] D. Lea, J. Ghawaly, G. G. Richard III, A. Ali-Gombe, A. Case, REx86: A local large language model for assisting in x86 assembly reverse engineering, in: Annual Computer Security Applications Conference (ACSAC), 2025, arXiv:2510.20975. doi:10.1109/acsac67867.2025.00024.

[34] W. Li, S. Manickam, Y.-w. Chong, S. Karuppayah, PhishDebate: An LLM-based multi-agent framework for phishing website detection, in: IEEE International Conference on Big Data (BigData), 2025, pp. 6606–6615, arXiv:2506.15656. The published citation is given in the arXiv Comments field. doi:10.1109/BigData66926.2025.11401440.

[35] D. Tran, D. Kiela, Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets, PREPRINT, not peer reviewed. arXiv:2604.02460. Kept because it says our null is not the first of its kind, and because the crossover figure it reports for our own model class has no reviewed equivalent we could verify. Bertalanič and Fortuna reach the same conclusion in a reviewed venue (2026). doi:10.48550/arXiv.2604.02460.

[36] B. Bertalanič, C. Fortuna, The cost of consensus: Isolated self-correction prevails over unguided homogeneous multi-agent debate, in: ACM Conference on AI and Agentic Systems, 2026, arXiv:2605.00914. doi:10.1145/3786335.3813137.

[37] R. Evertz, N. Risse, C. Neuer, N. Müller, S. Normann, et al., Chasing shadows: Pitfalls in LLM security research, in: Network and Distributed System Security Symposium (NDSS), 2026, arXiv:2512.09549. Living appendix at llmpitfalls.org. doi:10.14722/ndss.2026.241749.

[38] J. Ng, A. Milani Fard, Evaluating retrieval-augmented generation for explainable malware analysis, in: ACM SecDev, 2026, poster. arXiv:2605.03140.

[39] J. Metz, L. F. D. de Abreu, E. A. Cherman, M. C. Monard, On the estimation of predictive evaluation measure baselines for multi-label learning, in: Advances in Artificial Intelligence, IBERAMIA, Lecture Notes in Computer Science, 2012, pp. 189–198, crossref types LNCS proceedings as book-chapter by convention; the container title is the IBERAMIA 2012 proceedings. doi:10.1007/978-3-642-34654-5_20.

[40] J. Metz, N. Spolaôr, E. A. Cherman, M. C. Monard, Comparing published multi-label classifier performance measures to the ones obtained by a simple multi-label baseline classifier, PREPRINT, not peer reviewed. arXiv:1503.06952, v1 only, no venue. Kept for its systematic-review finding rather than for the requirement itself, which now rests on the reviewed IBERAMIA 2012 paper by the same group (2015). doi:10.48550/arXiv.1503.06952.

[41] A. Lazaridou, A. Kuncoro, E. Gribovskaya, et al., Mind the gap: Assessing temporal generalization in neural language models, in: Advances in Neural Information Processing Systems (NeurIPS), 2021, arXiv:2102.01951.

[42] T. Xu, J. Zhang, P. Huang, J. Zheng, T. Sheng, D. Yuan, Y. Zhou, S. Pasupathy, Do not blame users for misconfigurations, in: 24th ACM Symposium on Operating Systems Principles (SOSP), 2013, pp. 244–259, names the failure mode: "silent overruling refers to the case that the system changes an unacceptable user setting into the default value without notifying the user", measured across 74 parameters in Squid and Apache. doi:10.1145/2517349.2522727.

[43] O. Oyelayo, S. Abedu, S. Khatoonabadi, E. Shihab, Developer challenges in the adoption of model context protocol in AI agent development, in: 7th International Workshop on Bots and Agents in Software Engineering (BotSE), co-located with ICSE, 2026, pp. 53–59, bug reports and pull requests on MCP reference servers: data validation and type issues are the second most prevalent class at 13.97%. doi:10.1145/3786161.3788462.

[44] A. Martin-Lopez, S. Segura, A. Ruiz-Cortés, A catalogue of inter-parameter dependencies in RESTful web APIs, in: Service-Oriented Computing (ICSOC), 2019, pp. 399–414, across 40 APIs and over 2500 operations, input dependencies are "the norm, rather than the exception, with 85% of the reviewed APIs" having them, and OpenAPI cannot express them. doi:10.1007/978-3-030-33702-5_31.

[45] Y. Li, Z. Liu, P.-L. Poon, D. Towey, C.-A. Sun, et al., Metamorphic relation generation: State of the art and research directions, ACM Transactions on Software Engineering and MethodologyPreprint arXiv:2406.05397 (2024). doi:10.1145/3708521.

[46] S. Dev, A. Sloan, J. Kavner, N. Kong, M. Sandler, Judge reliability harness: Stress testing the reliability of LLM judges, in: Agents in the Wild: Safety, Security, and Beyond Workshop, International Conference on Learning Representations (ICLR), 2026, arXiv:2603.05399. Corrected: this paper does not establish the duplicate- item practice an earlier draft cited it for; it perturbs formatting, paraphrase, verbosity and labels.

[47] A. Ryan, S. S. Noor, M. Erfan, S. Mitra, S. Mittal, M. R. Rahman, Evaluating open-source LLMs for multi-label ATT&CK technique classification on CTI reports, PREPRINT, not peer reviewed. arXiv:2606.18166. The quoted correlation is from the full text, not the abstract: "the Spearman rank correlation confirms a statistically significant relationship between parameter size and F_1 scores in this CTI classification task (ρ=0.85, p=0.014)" (2026). doi:10.48550/arXiv.2606.18166.

[48] Y. Sun, H. Wang, J. Li, J. Liu, X. Li, H. Wen, Y. Yuan, H. Zheng, Y. Liang, Y. Li, Y. Liu, An empirical study of LLM reasoning ability under strict output

length constraint, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025, pp. 7652–7671, finding 3: under strict output constraints reasoning performance “might not scale monotonically with the model size”. Finding 4 independently covers our reasoning-flag confound: reasoning models do not always outperform instruction-tuned ones under the same constraint. `doi:10.18653/v1/2025.emnlp-main.389`.

[49] M. Mizrahi, G. Kaplan, D. Malkin, R. Dror, D. Shahaf, G. Stanovsky, State of what art? A call for multi-prompt LLM evaluation, Transactions of the Association for Computational Linguistics 12 (2024) 933–949, 6.5M instances over 20 models and 39 tasks: “different instruction templates lead to very different performance, both absolute and relative”, and the relative part is the ranking. `doi:10.1162/tacl_a_00681`.

[50] C. Snell, J. Lee, K. Xu, A. Kumar, Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning, in: International Conference on Learning Representations (ICLR), 2025, oral. “In a FLOPs-matched evaluation, we find that on problems where a smaller base model attains somewhat non-trivial success rates, test-time compute can be used to outperform a 14x larger model”.

[51] M. Hassid, T. Remez, J. Gehring, R. Schwartz, Y. Adi, The larger the better? Improved LLM code-generation via budget reallocation, in: First Conference on Language Modeling (COLM), 2024, the ranking flips and flips back on an inference-time choice alone: repeated small-model sampling gains up to 15% with a unit-test oracle, and loses to a single large-model output without one.

[52] M. Jin, Q. Yu, D. Shu, H. Zhao, W. Hua, Y. Meng, Y. Zhang, M. Du, The impact of reasoning step length on large language models, in: Findings of the Association for Computational Linguistics: ACL, 2024, pp. 1830–1842, cited as the counterweight: lengthening reasoning steps “considerably enhances LLMs’ reasoning abilities”. Read with Sun et al., the effect is task- and budget-dependent rather than monotone in either direction. `doi:10.18653/v1/2024.findings-acl.108`.

[53] `max_tokens` not respected on the non-chat endpoint, PREPRINT, not peer reviewed. ggml-org/llama.cpp issue #8634. Search result; the issue has not been read in full. Cited with the vllm issue to demote our own mechanism to a known integration wart, which is the conservative direction (2024).

[54] Request for a server-side output cap, PREPRINT, not peer reviewed. vllm-project/vllm issue #11976 (2024).

[55] D. Kumaran, Reported confidence in LLMs tracks commitment more than correctness, PREPRINT, not peer reviewed. arXiv:2606.29490, submitted 2026-06-28 and revised 2026-08-24. Verified against the arXiv listing and the export

API on 2026-08-24: a single author rather than the "et al." an internal note had recorded, no Comments field, no journal reference and no DOI beyond the arXiv one. Eight models over two classes, with the contrast that carries the claim being verbal confidence against calibrated token log-probabilities (2026). `doi:10.48550/arXiv.2606.29490`.

[56] abuse.ch, MalwareBazaar, Malware sample corpus. `https://bazaar.abuse.ch`, cited as the corpus the dynamic cohort was drawn from. Its terms permit research redistribution of individual samples but not of the corpus as a package (2026).

[57] The MITRE Corporation, MITRE ATT&CK enterprise, Knowledge base. `https://attack.mitre.org`, the family-to-technique `uses` relationships used here as per-sample ground truth are derived from the `mitre-attack/attack-stix-data` distribution, under the ATT&CK Terms of Use (2026).

[58] D. Küçük, F. Ertam, Replication package: the pipeline, the evaluation harnesses, the per-sample records and the derivation of every number reported, Source repository, under an open licence. `https://github.com/Root0ne/Maljan`, the repository is the only place the system carries a name. The paper does not: it calls the object of study the pipeline throughout, and a check refuses that name anywhere but this entry (2026).